Biometric Face Attendance System - Technical Project Report

1. EXECUTIVE ARCHITECTURE & STACK BREAKDOWN

Technology Stack

- **Face Detection**: OpenCV YuNet ONNX (`models/face_detection_yunet_2023mar.onnx`) - A lightweight, fast, and accurate CNN-based face detector capable of edge deployment.
- **Feature Extraction**: PyTorch MobileNetV2 - Generates 128-dimensional L2 normalized embeddings representing facial features.
- **Similarity Search**: FAISS (`IndexFlatL2` wrapped in `IndexIDMap2`) - Performs high-speed nearest-neighbor searches in the 128-d vector space. Distance threshold is strictly `<= 0.55` to prevent false positives, while a stricter `<= 0.15` is used during enrollment to prevent duplicate registrations.
- **Database Layer**: SQLite (`attendance.db`). Enforces strict schema:
    - `users`: `id` (INTEGER PK), `user_code` (TEXT UNIQUE), `name` (TEXT), `registered_date` (TEXT)
    - `attendance`: `entry_id` (INTEGER PK), `user_code` (TEXT), `name` (TEXT), `date` (YYYY-MM-DD), `time` (HH:MM:SS) with a `UNIQUE(user_code, date)` constraint to prevent double-punching.
- **Backend**: FastAPI & Uvicorn running on `127.0.0.1:8000`. Handles cross-origin requests, image decoding, model inference, FAISS querying, and SQLite transactions.
- **Frontend**: Next.js 16 (Turbopack, Tailwind CSS HUD) running on `localhost:3000`. Features a Cyberpunk/HUD aesthetic with live webcam feeds, real-time toast notifications, and dynamic metrics.
- **Desktop Client**: Python Tkinter GUI (`app.py`). Serves as a standalone alternative kiosk and admin dashboard with Treeview reporting and CSV exports.

Core Math & Logic

- **L2 Euclidean Distance**: The formula $d(p, q) = \sqrt{\sum_{i=1}^{n} (q_i - p_i)^2}$ is used by FAISS on L2-normalized vectors. Because the vectors are normalized, L2 distance is mathematically related to Cosine Similarity. A distance threshold of `0.55` ensures that only tightly clustered vectors (representing the same person) are matched. Random faces or noise will yield distances {{content}}gt; 0.8$.
- **Attendance Metrics**:
    - `Total` = Total registered users in the `users` table.
    - `Present` = Unique users who have a punch in the `attendance` table for the target date/month.
    - `Absent` = `Total` - `Present` (floored at 0).
    - `Attendance Rate %` = `(Present / Total) * 100` (rounded to 1 decimal place).

---

2. FILE-BY-FILE CODE & TECHNICAL WALKTHROUGH

`vector_index.py`

This module encapsulates the FAISS vector database logic. It manages loading, creating, adding, removing, and searching 128-d embeddings. The `IndexIDMap2` is crucial as it allows mapping arbitrary integer UIDs to vectors directly in FAISS.

```
import os
import faiss
import numpy as np

class VectorIndex:
    def __init__(self, index_path='models/face_index.faiss', dim=128):
        self.index_path = index_path
        self.dim = dim
        self._load_or_create_index()

    def _load_or_create_index(self):
        if os.path.exists(self.index_path):
            try:
                self.index = faiss.read_index(self.index_path)
                print(f"[VectorIndex] Loaded existing index from {self.index_path} with {self.index.ntotal} vectors.")
            except Exception as e:
                print(f"[VectorIndex] Failed to load index: {e}. Creating a new one.")
                self._create_new_index()
        else:
            self._create_new_index()

    def _create_new_index(self):
        # Create a new IndexIDMap2 wrapping an IndexFlatL2
        # IndexIDMap2 is required to store integer IDs alongside vectors natively in FAISS
        flat_index = faiss.IndexFlatL2(self.dim)
        self.index = faiss.IndexIDMap2(flat_index)
        print(f"[VectorIndex] Initialized fresh FAISS IndexIDMap2(IndexFlatL2({self.dim})).")
        self.save()

    def add_vector(self, user_id: int, embedding: np.ndarray):
        """
        Normalizes and adds a 128-dimensional embedding to the FAISS index.
        """
        # Ensure embedding is 2D array of shape (1, 128) and float32
        if embedding.ndim == 1:
            embedding = np.expand_dims(embedding, axis=0)

        embedding = np.ascontiguousarray(embedding, dtype=np.float32)

        if embedding.shape[1] != self.dim:
            raise ValueError(f"Embedding dimension mismatch. Expected {self.dim}, got {embedding.shape[1]}")
```

```python
        # Normalize the embedding using L2 norm
        faiss.normalize_L2(embedding)

        # FAISS expects IDs as int64
        id_array = np.array([user_id], dtype=np.int64)

        # Add to index
        self.index.add_with_ids(embedding, id_array)
        self.save()
        return True

    def remove_vector(self, user_id: int):
        """Removes the vector associated with the given user_id from the index."""
        if self.index.ntotal == 0:
            return False
        selector = faiss.IDSelectorArray(np.array([user_id], dtype=np.int64))
        self.index.remove_ids(selector)
        self.save()
        return True

    def search(self, query_emb: np.ndarray, threshold=0.55):
        """
        Searches the index for the closest vector to query_emb.
        Returns (matched_user_id, distance) if successful, else (None, distance).
        """
        if self.index.ntotal == 0:
            return None, float('inf')

        # Ensure query is 2D array and float32
        if query_emb.ndim == 1:
            query_emb = np.expand_dims(query_emb, axis=0)

        query_emb = np.ascontiguousarray(query_emb, dtype=np.float32)

        # Normalize the query embedding for cosine similarity equivalent using L2
        faiss.normalize_L2(query_emb)

        # Search for top 1 closest vector
        distances, indices = self.index.search(query_emb, k=1)

        matched_user_id = int(indices[0][0])
        distance = float(distances[0][0])

        if matched_user_id != -1 and distance <= threshold:
            return matched_user_id, distance

        return None, distance

    def save(self):
        """Saves the current index to disk."""
        os.makedirs(os.path.dirname(self.index_path), exist_ok=True)
        faiss.write_index(self.index, self.index_path)

if __name__ == '__main__':
    print("=== STEP 3: vector_index.py SELF-TEST ===")

    # Use a dummy path for the self-test
    test_index_path = 'models/test_face_index.faiss'
    vi = VectorIndex(index_path=test_index_path, dim=128)

    # 1. Create a mock embedding and add to index
    mock_id = 999
    np.random.seed(42)
    mock_emb = np.random.randn(128).astype(np.float32)

    print(f"1. Adding mock vector for User ID {mock_id}...")
    vi.add_vector(mock_id, mock_emb)
    print(f"   [OK] Vector added. Total vectors in index: {vi.index.ntotal}")

    # 2. Perform a search query with the exact same embedding
    print(f"2. Querying the exact same vector...")
    found_id, dist = vi.search(mock_emb.copy(), threshold=0.55)

    if found_id == mock_id:
        print(f"   [OK] Match successful! Found ID {found_id} with distance {dist:.5f}")
    else:
        print(f"   [FAIL] Search returned ID {found_id} with distance {dist:.5f}")

    # 3. Query with random noise (should fail the threshold)
    print(f"3. Querying with random noise...")
    noise_emb = np.random.randn(128).astype(np.float32)
    found_id_noise, dist_noise = vi.search(noise_emb, threshold=0.55)

    if found_id_noise is None:
        print(f"   [OK] Noise successfully rejected. Distance {dist_noise:.5f} > threshold 0.55")
```

```
    else:
        print(f"   [FAIL] Noise was incorrectly matched to ID {found_id_noise} with distance {dist_noise:.5f}")

    # Clean up test index
    if os.path.exists(test_index_path):
        os.remove(test_index_path)

    print("=== SELF-TEST COMPLETE ===")
```

Explanation:

- **Initialization**: Attempts to load an existing `.faiss` index from disk. If absent or corrupted, initializes a fresh `IndexFlatL2` wrapped in `IndexIDMap2`.
- **`add_vector`**: Normalizes the incoming 128-d vector via `faiss.normalize_L2` and maps it to the `user_id` using `add_with_ids`.
- **`remove_vector`**: Creates an `IDSelectorArray` targeting the given UID, safely purging the exact vector without disturbing the rest of the index, and saves state.
- **`search`**: Computes the L2 distance between the normalized query vector and the index. If the closest match distance is less than or equal to the threshold (0.55), returns the UID.

`backend_api.py`

This FastAPI application serves as the bridge between the frontend interfaces and the underlying PyTorch/FAISS/SQLite engine.

```
import base64
import cv2
import numpy as np
from fastapi import FastAPI, HTTPException, Response
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from typing import List
import io
import csv
import time
import sqlite3
from datetime import datetime

import db_manager as db
from face_engine import FaceEngine
from vector_index import VectorIndex

app = FastAPI(title="Face Attendance AI Engine")

# Next.js requests allow karne ke liye CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

# AI Models & Index initialization
db.init_db()
engine = FaceEngine()
v_index = VectorIndex()

class FramePayload(BaseModel):
    image_base64: str

class EnrollPayload(BaseModel):
    user_code: str
    name: str
    images_base64: List[str]

class LoginPayload(BaseModel):
    username: str
    password: str

class UserUpdateSchema(BaseModel):
    user_code: str
    name: str

def parse_base64_image(b64_string: str):
    if "," in b64_string:
        b64_string = b64_string.split(",")[1]
    raw_bytes = base64.b64decode(b64_string)
    np_arr = np.frombuffer(raw_bytes, np.uint8)
    return cv2.imdecode(np_arr, cv2.IMREAD_COLOR)

# 1. Kiosk Attendance Endpoint
@app.post("/api/scan-attendance")
def scan_attendance(payload: FramePayload):
    try:
        frame = parse_base64_image(payload.image_base64)
        if frame is None:
            return {"status": "error", "message": "Corrupted frame"}

        faces = engine.detect_faces(frame)
```

```
        if faces is None or len(faces) == 0:
            return {"status": "no_face", "message": "No face detected in camera viewport"}

        # Extract embedding
        x, y, w, h = [int(v) for v in faces[0][:4]]
        face_crop = frame[max(0, y):max(0, y)+h, max(0, x):max(0, x)+w]
        emb = engine.extract_embedding(face_crop)

        # L2 Normalize embedding
        norm = np.linalg.norm(emb)
        emb = emb / (norm + 1e-10)

        # FAISS Vector Search (Attendance threshold <= 0.42)
        matched_id, dist = v_index.search(emb, threshold=0.42)

        if matched_id is not None:
            user = db.get_user_by_id(matched_id)
            if user:
                import sqlite3
                conn = sqlite3.connect(db.DB_PATH)
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM attendance WHERE user_code = ? AND date = DATE('now', 'localtime')", (user[0],))
                already_marked = cursor.fetchone()
                conn.close()

                if already_marked:
                    return {
                        "status": "already_marked",
                        "name": user[1],
                        "user_code": user[0],
                        "uid": matched_id
                    }

                success, info = db.mark_attendance(user[0], user[1])
                if success:
                    return {
                        "status": "success",
                        "uid": matched_id,
                        "user_code": user[0],
                        "name": user[1],
                        "distance": round(float(dist), 4)
                    }
                else:
                    return {
                        "status": "already_marked",
                        "name": user[1],
                        "user_code": user[0],
                        "uid": matched_id
                    }

        return {"status": "unknown"}
    except Exception as e:
        print(f"Error processing frame: {e}")
        return {"status": "error", "message": "Invalid frame"}

# 2. Get Registered Users Endpoint
@app.get("/api/users")
def get_users():
    raw_users = db.get_all_users()
    return [{"uid": u[0], "code": u[1], "name": u[2], "date": u[3], "status": "Active"} for u in raw_users]

# 3. Enrollment Endpoint (Strict 0.15 Duplicate Collision Check)
@app.post("/api/enroll")
def enroll_user(payload: EnrollPayload):
    if len(payload.images_base64) == 0:
        raise HTTPException(status_code=400, detail="No images provided.")

    embeddings = []
    for b64 in payload.images_base64:
        frame = parse_base64_image(b64)
        if frame is None:
            continue
        faces = engine.detect_faces(frame)
        if faces is not None and len(faces) > 0:
            face = max(faces, key=lambda f: f[2]*f[3])
            x, y, w, h = face[:4].astype(int)
            pad_x = int(w * 0.1)
            pad_y = int(h * 0.1)
            x1 = max(0, x - pad_x)
            y1 = max(0, y - pad_y)
            x2 = min(frame.shape[1], x + w + pad_x)
            y2 = min(frame.shape[0], y + h + pad_y)
            face_crop = frame[y1:y2, x1:x2]
            if face_crop.shape[0] > 0 and face_crop.shape[1] > 0:
                emb = engine.extract_embedding(face_crop)
                embeddings.append(emb)
```

```
    if len(embeddings) == 0:
        raise HTTPException(status_code=400, detail="No valid faces detected in the provided samples.")

    # Calculate mean embedding
    mean_emb = np.mean(embeddings, axis=0).astype(np.float32)
    norm = np.linalg.norm(mean_emb)
    if norm > 0:
        mean_emb = mean_emb / norm

    # Strict Duplicate Prevention (Threshold 0.15 to match app.py)
    matched_id, dist = v_index.search(mean_emb, threshold=0.15)
    if matched_id is not None:
        user = db.get_user_by_id(matched_id)
        if user:
            raise HTTPException(
                status_code=409,
                detail=f"Duplicate biometric detected! Already enrolled as {user[1]} (UID #{matched_id})"
            )

    reg_date = time.strftime("%Y-%m-%d")
    user_id = db.add_user(payload.user_code, payload.name, reg_date)
    if user_id == -1:
         raise HTTPException(status_code=409, detail="User Code already exists!")

    v_index.add_vector(user_id, mean_emb)
    return {"status": "success", "uid": user_id, "name": payload.name}

# 4. Export Endpoints
@app.get("/api/export/today")
def export_today():
    records = db.get_today_attendance()
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(["User Code", "Name", "Date", "Time"])
    writer.writerows(records)

    return Response(
        content=output.getvalue(),
        media_type="text/csv",
        headers={"Content-Disposition": f"attachment; filename=Attendance_{time.strftime('%Y-%m-%d')}.csv"}
    )

@app.get("/api/export/summary")
def export_summary():
    summary = db.get_user_attendance_summary()
    output = io.StringIO()
    writer = csv.writer(output)
    writer.writerow(["User Code", "Name", "Total Days", "Present", "Absent", "Attendance %"])
    for s in summary:
        writer.writerow([s['user_code'], s['name'], s['total_days'], s['present'], s['absent'], s['percentage']])

    return Response(
        content=output.getvalue(),
        media_type="text/csv",
        headers={"Content-Disposition": f"attachment; filename=Attendance_Summary_{time.strftime('%Y-%m-%d')}.csv"}
    )

# 5. Admin Auth Endpoint
@app.post("/api/admin/login")
def admin_login(payload: LoginPayload):
    if db.verify_admin(payload.username, payload.password):
        return {"status": "success"}
    else:
        raise HTTPException(status_code=401, detail="Invalid Credentials")

@app.get("/api/reports/daily")
def get_daily_report(date_str: str = None):
    return db.get_daily_report(date_str)

@app.get("/api/attendance-logs")
def get_attendance_logs():
    records = db.get_today_attendance()
    return [{"id": idx, "code": r[0], "name": r[1], "time": r[3]} for idx, r in enumerate(records)]

@app.get("/api/reports/monthly")
def get_monthly_report(month_str: str = None):
    return db.get_monthly_report(month_str)

@app.put("/api/users/{user_id}")
def update_user(user_id: int, payload: UserUpdateSchema):
    success, msg = db.update_user(user_id, payload.user_code, payload.name)
    if success:
        return {"status": "success", "message": msg}
    else:
        raise HTTPException(status_code=400, detail=msg)
```

```
@app.delete("/api/users/{user_id}")
def delete_user(user_id: int):
    # Remove from FAISS index first
    v_index.remove_vector(user_id)
    # Remove from SQLite
    success, msg = db.delete_user(user_id)
    if success:
        return {"status": "success", "message": "User deleted and FAISS index rebuilt"}
    else:
        raise HTTPException(status_code=400, detail=msg)

@app.get("/api/reports/user-monthly")
def get_user_monthly_report(user_id: int, month_str: str = None):
    res = db.get_user_monthly_report(user_id, month_str)
    if res.get("status") == "error":
        raise HTTPException(status_code=404, detail=res["message"])
    return res

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("backend_api:app", host="0.0.0.0", port=8000, reload=True)
```

Explanation:

- **`/api/scan-attendance`**: Decodes base64 frame, detects face, extracts embedding, queries FAISS. If matched, checks SQLite for duplicate daily punches. If clear, logs attendance and returns `success`.
- **`/api/enroll`**: Processes 5 base64 frames, extracts embeddings, computes the mean vector, normalizes it, and checks FAISS with a strict `0.15` threshold for duplicates. If clear, saves to SQLite and FAISS.
- **User Management Endpoints**: `PUT /api/users/{user_id}` and `DELETE /api/users/{user_id}` safely update or purge user data across both SQLite and FAISS.
- **Reporting Endpoints**: Daily, Monthly, and Individual Monthly endpoints query SQLite to generate JSON aggregation metrics for the frontend.

`db_manager.py`

This module serves as the primary Data Access Layer (DAL). It interfaces with the SQLite `attendance.db` database to handle user management, attendance logging, and reporting queries.

```
import sqlite3
import hashlib
from datetime import datetime

DB_PATH = "attendance.db"

def init_db():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS admins (
            username TEXT PRIMARY KEY,
            password_hash TEXT NOT NULL
        )
    ''')

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_code TEXT UNIQUE NOT NULL,
            name TEXT NOT NULL,
            registered_date TEXT NOT NULL
        )
    ''')

    cursor.execute('''
        CREATE TABLE IF NOT EXISTS attendance (
            entry_id INTEGER PRIMARY KEY AUTOINCREMENT,
            user_code TEXT,
            name TEXT,
            date TEXT,
            time TEXT,
            UNIQUE(user_code, date)
        )
    ''')

    cursor.execute("SELECT * FROM admins WHERE username = 'admin'")
    if not cursor.fetchone():
        hashed_pw = hashlib.sha256("admin123".encode()).hexdigest()
        cursor.execute("INSERT INTO admins (username, password_hash) VALUES (?, ?)", ("admin", hashed_pw))

    conn.commit()
    conn.close()

def verify_admin(username, password) -> bool:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    hashed_pw = hashlib.sha256(password.encode()).hexdigest()
```

```
    cursor.execute("SELECT * FROM admins WHERE username = ? AND password_hash = ?", (username, hashed_pw))
    result = cursor.fetchone() is not None
    conn.close()
    return result

def add_user(user_code, name, reg_date=None) -> int:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    if not reg_date:
        reg_date = datetime.now().strftime('%Y-%m-%d')

    try:
        cursor.execute("INSERT INTO users (user_code, name, registered_date) VALUES (?, ?, ?)",
                       (user_code, name, reg_date))
        conn.commit()
        numeric_id = cursor.lastrowid
        return numeric_id
    except sqlite3.IntegrityError:
        return -1
    finally:
        conn.close()

def update_user(numeric_id, new_user_code, new_name, new_reg_date=None):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    try:
        cursor.execute("SELECT user_code FROM users WHERE id = ?", (numeric_id,))
        row = cursor.fetchone()
        if not row:
            return False, "User not found."

        old_user_code = row[0]

        if new_reg_date:
            cursor.execute("UPDATE users SET user_code = ?, name = ?, registered_date = ? WHERE id = ?",
                           (new_user_code, new_name, new_reg_date, numeric_id))
        else:
            cursor.execute("UPDATE users SET user_code = ?, name = ? WHERE id = ?",
                           (new_user_code, new_name, numeric_id))

        cursor.execute("UPDATE attendance SET user_code = ?, name = ? WHERE user_code = ?",
                       (new_user_code, new_name, old_user_code))

        conn.commit()
        return True, "User updated successfully."
    except sqlite3.IntegrityError:
        return False, "User code already exists."
    except Exception as e:
        return False, str(e)
    finally:
        conn.close()

def delete_user(numeric_id: int):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    try:
        cursor.execute("SELECT user_code FROM users WHERE id = ?", (numeric_id,))
        row = cursor.fetchone()
        if not row:
            return False, "User not found."

        user_code = row[0]

        cursor.execute("DELETE FROM users WHERE id = ?", (numeric_id,))
        cursor.execute("DELETE FROM attendance WHERE user_code = ?", (user_code,))

        conn.commit()
        return True, "User deleted successfully."
    except Exception as e:
        return False, str(e)
    finally:
        conn.close()

def get_all_users() -> list:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("SELECT id, user_code, name, registered_date FROM users ORDER BY id ASC")
    results = cursor.fetchall()
    conn.close()
    return results

def get_user_by_id(numeric_id) -> tuple:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("SELECT user_code, name FROM users WHERE id = ?", (numeric_id,))
```

```
    result = cursor.fetchone()
    conn.close()
    return result

def mark_attendance(user_code, name):
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    now = datetime.now()
    date_str = now.strftime('%Y-%m-%d')
    time_str = now.strftime('%H:%M:%S')

    try:
        cursor.execute("INSERT INTO attendance (user_code, name, date, time) VALUES (?, ?, ?, ?)",
                       (user_code, name, date_str, time_str))
        conn.commit()
        return True, time_str
    except sqlite3.IntegrityError:
        # Already punched in today
        # Fetch the existing time
        cursor.execute("SELECT time FROM attendance WHERE user_code = ? AND date = ?", (user_code, date_str))
        existing_time = cursor.fetchone()[0]
        return False, existing_time
    finally:
        conn.close()

def get_today_attendance() -> list:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    date_str = datetime.now().strftime('%Y-%m-%d')
    cursor.execute("SELECT user_code, name, date, time FROM attendance WHERE date = ? ORDER BY entry_id DESC", (date_str,))
    results = cursor.fetchall()
    conn.close()
    return results

def get_daily_attendance_stats() -> dict:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    date_str = datetime.now().strftime('%Y-%m-%d')

    cursor.execute("SELECT COUNT(id) FROM users")
    total = cursor.fetchone()[0] or 0

    cursor.execute("SELECT COUNT(entry_id) FROM attendance WHERE date = ?", (date_str,))
    present = cursor.fetchone()[0] or 0

    absent = max(0, total - present)
    percentage = round((present / total * 100), 1) if total > 0 else 0.0

    conn.close()
    return {'total': total, 'present': present, 'absent': absent, 'percentage': percentage}

def get_user_attendance_summary() -> list:
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("SELECT COUNT(DISTINCT date) FROM attendance")
    total_working_days = cursor.fetchone()[0] or 0

    cursor.execute("SELECT id, user_code, name FROM users ORDER BY id ASC")
    users = cursor.fetchall()

    summary = []
    for u in users:
        uid, user_code, name = u
        cursor.execute("SELECT COUNT(DISTINCT date) FROM attendance WHERE user_code = ?", (user_code,))
        present_days = cursor.fetchone()[0] or 0
        absent_days = max(0, total_working_days - present_days)
        percentage = f"{round((present_days / total_working_days * 100), 1)}%" if total_working_days > 0 else "0.0%"

        summary.append({
            'id': uid,
            'user_code': user_code,
            'name': name,
            'total_days': total_working_days,
            'present': present_days,
            'absent': absent_days,
            'percentage': percentage
        })

    conn.close()
    return summary

def get_daily_report(date_str=None) -> dict:
    if not date_str:
        date_str = datetime.now().strftime("%Y-%m-%d")
```

```
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("""
        SELECT u.id, u.user_code, u.name, a.time
        FROM users u
        LEFT JOIN attendance a ON u.user_code = a.user_code AND a.date = ?
        ORDER BY u.id ASC
    """, (date_str,))
    rows = cursor.fetchall()
    conn.close()

    records = []
    present_count = 0
    for r in rows:
        is_present = r[3] is not None
        if is_present:
            present_count += 1
        records.append({
            "uid": r[0],
            "user_code": r[1],
            "name": r[2],
            "date": date_str,
            "punch_time": r[3] if is_present else "--",
            "status": "Present" if is_present else "Absent"
        })

    total_users = len(records)
    rate = round((present_count / total_users * 100), 1) if total_users > 0 else 0.0

    return {
        "date": date_str,
        "total_users": total_users,
        "present_count": present_count,
        "absent_count": total_users - present_count,
        "attendance_rate": rate,
        "records": records
    }

def get_monthly_report(month_str=None) -> dict:
    if not month_str:
        month_str = datetime.now().strftime("%Y-%m")

    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    cursor.execute("""
        SELECT COUNT(DISTINCT date)
        FROM attendance
        WHERE strftime('%Y-%m', date) = ?
    """, (month_str,))
    total_days_conducted = cursor.fetchone()[0] or 1

    cursor.execute("""
        SELECT u.id, u.user_code, u.name,
               COUNT(DISTINCT a.date) as days_present
        FROM users u
        LEFT JOIN attendance a ON u.user_code = a.user_code AND strftime('%Y-%m', a.date) = ?
        GROUP BY u.id
        ORDER BY u.id ASC
    """, (month_str,))
    rows = cursor.fetchall()
    conn.close()

    records = []
    for r in rows:
        p_days = r[3]
        rate = round((p_days / total_days_conducted * 100), 1)
        records.append({
            "uid": r[0],
            "user_code": r[1],
            "name": r[2],
            "month": month_str,
            "days_present": p_days,
            "days_absent": max(0, total_days_conducted - p_days),
            "total_days": total_days_conducted,
            "attendance_percentage": rate
        })

    return {
        "month": month_str,
        "working_days": total_days_conducted,
        "records": records
    }

def get_user_monthly_report(numeric_id: int, month_str=None) -> dict:
```

```
    if not month_str:
        month_str = datetime.now().strftime("%Y-%m")

    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()

    # Get user details
    cursor.execute("SELECT id, user_code, name FROM users WHERE id = ?", (numeric_id,))
    user = cursor.fetchone()
    if not user:
        conn.close()
        return {"status": "error", "message": "User not found"}

    # Total unique active days conducted in this month
    cursor.execute("SELECT COUNT(DISTINCT date) FROM attendance WHERE strftime('%Y-%m', date) = ?", (month_str,))
    total_working = cursor.fetchone()[0] or 0
    effective_days = max(1, total_working)

    # Day-by-day punches of this specific user in this month
    cursor.execute("""
        SELECT date, time
        FROM attendance
        WHERE user_code = ? AND strftime('%Y-%m', date) = ?
        ORDER BY date ASC, time ASC
    """, (user[1], month_str))
    punches = cursor.fetchall()
    conn.close()

    # We want unique days present
    days_present = len(set([p[0] for p in punches]))
    attendance_rate = round((days_present / effective_days * 100), 1) if total_working > 0 else 0.0

    return {
        "status": "success",
        "uid": user[0],
        "user_code": user[1],
        "name": user[2],
        "month": month_str,
        "total_working_days": total_working,
        "days_present": days_present,
        "days_absent": max(0, total_working - days_present),
        "attendance_rate": attendance_rate,
        "daily_breakdown": [{"date": p[0], "punch_time": p[1], "status": "Present"} for p in punches]
    }


if __name__ == '__main__':
    print("=== STEP 2: db_manager.py SELF-TEST ===")

    print("1. Initializing DB...")
    init_db()

    print("2. Verifying default admin (admin / admin123)...")
    if verify_admin("admin", "admin123"):
        print("  [OK] Admin verification successful.")
    else:
        print("  [FAIL] Admin verification failed.")

    print("3. Testing user creation...")
    uid = add_user("TEST_001", "Test User")
    if uid != -1:
        print(f"  [OK] User created successfully with ID: {uid}.")
    else:
        print("  [INFO] User already exists.")

    print("4. Testing attendance marking...")
    success, t = mark_attendance("TEST_001", "Test User")
    if success:
        print(f"  [OK] Attendance marked successfully at {t}.")
    else:
        print(f"  [INFO] Attendance already marked today at {t}.")

    print("5. Running stats queries...")
    stats = get_daily_attendance_stats()
    print(f"  [OK] Daily Stats: {stats}")

    summary = get_user_attendance_summary()
    print(f"  [OK] User Summary retrieved. Found {len(summary)} user(s).")

    print("=== SELF-TEST COMPLETE ===")
```

Explanation:

- **`init_db`**: Creates the strict schema (`admins`, `users`, `attendance`) if it doesn't exist and seeds a default admin.
- **`add_user` / `update_user` / `delete_user`**: Handles DML operations for the `users` table. `update_user` cascades user code changes to the `attendance` table.

`delete_user` permanently scrubs both user and historical logs.

- **`mark_attendance`**: Relies on SQLite's `UNIQUE(user_code, date)` constraint to seamlessly prevent double-punching natively.
- **Reporting Queries**: Contains aggregations for daily stats (`get_daily_attendance_stats`), specific day reports (`get_daily_report`), and complex monthly breakdowns (`get_monthly_report`, `get_user_monthly_report`).

`face_engine.py`

The core Computer Vision logic encapsulating OpenCV's YuNet for ultra-fast face detection and PyTorch's MobileNetV2 for feature extraction.

```
import os
import cv2
cv2.utils.logging.setLogLevel(cv2.utils.logging.LOG_LEVEL_ERROR)
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision.models as models
from torchvision import transforms
from PIL import Image

class MobileNetV2Embedder(nn.Module):
    def __init__(self):
        super(MobileNetV2Embedder, self).__init__()
        # Load base model without weights
        base_model = models.mobilenet_v2(weights=None)

        # Feature extractor
        self.features = base_model.features

        # Pooling layer
        self.pool = nn.AdaptiveAvgPool2d((1, 1))

        # Custom head
        self.head = nn.Sequential(
            nn.Dropout(p=0.2),
            nn.Linear(base_model.last_channel, 256),
            nn.ReLU(),
            nn.Linear(256, 128)
        )

    def forward(self, x):
        # Extract features
        x = self.features(x)
        # Apply adaptive pooling
        x = self.pool(x)
        # Flatten
        x = torch.flatten(x, 1)
        # Pass through head
        x = self.head(x)
        # Apply L2 normalization
        x = F.normalize(x, p=2, dim=1)
        return x


class FaceEngine:
    def __init__(self,
                 detector_path='models/face_detection_yunet_2023mar.onnx',
                 embedder_path='models/mobilenetv2_feature_extractor.pth'):
        # Initialize device strictly on CPU
        self.device = torch.device("cpu")

        # Load YuNet Detector
        if not os.path.exists(detector_path):
            print(f"[Warning] Detector model not found at {detector_path}. Detection will fail if called.")
            self.detector = None
        else:
            self.detector = cv2.FaceDetectorYN.create(
                detector_path,
                "",
                (640, 480),
                score_threshold=0.8,
                nms_threshold=0.3
            )

        # Load MobileNetV2 Embedder
        self.embedder = MobileNetV2Embedder()
        if os.path.exists(embedder_path):
            self.embedder.load_state_dict(torch.load(embedder_path, map_location=self.device))
        else:
            print(f"[Warning] Embedder weights not found at {embedder_path}. Using random weights.")

        self.embedder.to(self.device)
        self.embedder.eval()

        # Image Preprocessing Transform
        self.transform = transforms.Compose([
```

```
            transforms.Resize((160, 160)),
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
        ])

    def detect_faces(self, frame_bgr):
        if self.detector is None:
            raise RuntimeError("Detector model was not loaded properly.")

        # Dynamically set input size based on current frame shape
        h, w = frame_bgr.shape[:2]
        self.detector.setInputSize((w, h))

        # Detect faces
        _, faces = self.detector.detect(frame_bgr)
        return faces

    def extract_embedding(self, face_crop_bgr) -> np.ndarray:
        # Convert BGR crop to RGB
        face_crop_rgb = cv2.cvtColor(face_crop_bgr, cv2.COLOR_BGR2RGB)

        # Create PIL Image
        pil_img = Image.fromarray(face_crop_rgb)

        # Apply transforms
        tensor_img = self.transform(pil_img).unsqueeze(0).to(self.device)

        # Run forward pass through the embedder
        with torch.no_grad():
            emb = self.embedder(tensor_img)

        # Return a 128-dimensional float32 NumPy array
        return emb.squeeze().cpu().numpy().astype(np.float32)


if __name__ == '__main__':
    print("=== STEP 4: face_engine.py SELF-TEST ===")

    try:
        # 1. Initialize FaceEngine
        engine = FaceEngine()

        # 2. Feed a synthetic mock frame through detection
        mock_frame = np.random.randint(0, 256, (480, 640, 3), dtype=np.uint8)
        print("Feeding mock frame of shape:", mock_frame.shape)
        if engine.detector is not None:
            faces = engine.detect_faces(mock_frame)
            print("Detected faces:", faces if faces is not None else "None")
        else:
            print("Skipped face detection test due to missing model.")

        # 3. Run a dummy face crop through extract_embedding
        mock_face_crop = np.random.randint(0, 256, (112, 112, 3), dtype=np.uint8)
        embedding = engine.extract_embedding(mock_face_crop)

        # 4. Confirm the output shape is (128,)
        print(f"Embedding shape: {embedding.shape}")
        print(f"Embedding dtype: {embedding.dtype}")

        if embedding.shape == (128,):
            print("[OK] Embedding extraction successful!")
        else:
            print("[ERROR] Embedding shape mismatch.")

    except Exception as e:
        print(f"[ERROR] Self-test failed: {e}")
    print("=== SELF-TEST COMPLETE ===")
```

Explanation:

- **YuNet Initialization**: Loads the lightweight `.onnx` detector optimized for edge execution.
- **MobileNetV2 Initialization**: Loads a pre-trained PyTorch MobileNetV2 model and strips the classification head to output raw 128-d feature embeddings.
- **`detect_faces`**: Runs YuNet over an image array, returning bounding boxes and landmarks for all detected faces.
- **`extract_embedding`**: Takes a cropped face array, resizes it to 160x160, normalizes the RGB channels, passes it through the PyTorch tensor pipeline, and detaches the resulting 128-d feature vector for FAISS consumption.

`frontend/next.config.ts`

Configuration file for Next.js.

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  /* config options here */
};

export default nextConfig;
```

Explanation:

- Standard Turbopack configuration. Currently empty as CORS is fully handled by FastAPI (`CORSMiddleware` with `allow_origins=["*"]`), preventing fetch drops without needing proxy rewrites here.

`frontend/app/page.tsx`

The primary React/Next.js Client Component powering the Cyberpunk HUD interface.

```
"use client";

import React, { useState, useEffect, useRef } from 'react';
import Webcam from 'react-webcam';
import {
  Camera,
  Users,
  Activity,
  Eye,
  UserCheck,
  ShieldCheck,
  Clock,
  Search,
  UserPlus,
  Trash2,
  Edit3,
  Download,
  CheckCircle2,
  AlertTriangle,
  RefreshCw,
  Sparkles
} from 'lucide-react';

export default function Home() {
  const [activeTab, setActiveTab] = useState<'kiosk' | 'directory'>('kiosk');
  const [isScanning, setIsScanning] = useState<boolean>(true);
  const [verifiedToast, setVerifiedToast] = useState<boolean>(false);
  const [searchQuery, setSearchQuery] = useState<string>('');
  const [isCameraOn, setIsCameraOn] = useState<boolean>(true);
  const [scanResult, setScanResult] = useState<any>(null);
  const [hudStatus, setHudStatus] = useState<string>("Ready to Scan");
  const [hudType, setHudType] = useState<'neutral' | 'success' | 'error' | 'warning'>('neutral');

  const [isAdminAuth, setIsAdminAuth] = useState<boolean>(false);
  const [showAdminLogin, setShowAdminLogin] = useState<boolean>(false);
  const [adminUser, setAdminUser] = useState<string>('');
  const [adminPass, setAdminPass] = useState<string>('');
  const [enrollStatus, setEnrollStatus] = useState<string>('');
  const [isEnrolling, setIsEnrolling] = useState<boolean>(false);

  const [reportTab, setReportTab] = useState<'directory' | 'daily' | 'monthly'>('directory');
  const [dailyDate, setDailyDate] = useState<string>(new Date().toISOString().split('T')[0]);
  const [monthlyMonth, setMonthlyMonth] = useState<string>(new Date().toISOString().slice(0, 7));
  const [dailyReport, setDailyReport] = useState<any>(null);
  const [monthlyReport, setMonthlyReport] = useState<any>(null);

  const [editingUser, setEditingUser] = useState<any>(null);
  const [editCode, setEditCode] = useState('');
  const [editName, setEditName] = useState('');

  const [individualReportUser, setIndividualReportUser] = useState<any>(null);
  const [individualReport, setIndividualReport] = useState<any>(null);
  const [individualReportMonth, setIndividualReportMonth] = useState<string>(new Date().toISOString().slice(0, 7));

  // Registered database from backend
  const [users, setUsers] = useState<any[]>([]);

  useEffect(() => {
    fetch('http://127.0.0.1:8000/api/users')
      .then(res => res.json())
      .then(data => setUsers(data))
      .catch(err => setUsers([]));
  }, []);

  useEffect(() => {
    if (activeTab === 'directory' && reportTab === 'daily') {
      fetch(`http://127.0.0.1:8000/api/reports/daily?date_str=${dailyDate}`)
        .then(res => res.json())
        .then(data => setDailyReport(data))
```

```
      .catch(err => console.error(err));
  }
}, [activeTab, reportTab, dailyDate]);

useEffect(() => {
  if (activeTab === 'directory' && reportTab === 'monthly') {
    fetch(`http://127.0.0.1:8000/api/reports/monthly?month_str=${monthlyMonth}`)
      .then(res => res.json())
      .then(data => setMonthlyReport(data))
      .catch(err => console.error(err));
  }
}, [activeTab, reportTab, monthlyMonth]);

useEffect(() => {
  if (individualReportUser) {
    fetch(`http://127.0.0.1:8000/api/reports/user-monthly?user_id=${individualReportUser.uid}&month_str=${individualReportMonth}`)
      .then(res => res.json())
      .then(data => setIndividualReport(data))
      .catch(err => console.error(err));
  }
}, [individualReportUser, individualReportMonth]);

// Attendance punch feed
const [logs, setLogs] = useState<any[]>([]);

useEffect(() => {
  fetch('http://127.0.0.1:8000/api/attendance-logs')
    .then(res => res.json())
    .then(data => {
      if (Array.isArray(data)) {
        setLogs(data);
      }
    })
    .catch(err => console.error("Failed to load today's check-ins:", err));
}, []);

const presentToday = new Set(logs.map(log => log.code)).size;
const absentToday = Math.max(0, users.length - presentToday);
const attendanceRate = users.length > 0 ? Math.round((presentToday / users.length) * 100) : 0;

const webcamRef = useRef<Webcam>(null);
const enrollWebcamRef = useRef<Webcam>(null);
const [enrollCode, setEnrollCode] = useState('');
const [enrollName, setEnrollName] = useState('');

const toggleCamera = () => {
  if (isCameraOn && webcamRef.current && webcamRef.current.video) {
    const stream = webcamRef.current.video.srcObject as MediaStream;
    if (stream) {
      stream.getTracks().forEach(track => track.stop());
    }
  }
  setIsCameraOn(!isCameraOn);
};

// Live scan trigger
const handleLiveScan = async () => {
  if (!webcamRef.current) {
    setHudStatus("Camera not ready");
    setHudType('error');
    setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
    return;
  }

  setHudStatus("Analyzing Biometrics...");
  setHudType('neutral');

  const imageSrc = webcamRef.current?.getScreenshot();
  if (!imageSrc) {
    console.warn("Webcam not ready");
    setHudStatus("Camera not ready");
    setHudType('error');
    setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
    return;
  }

  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 5000);

  try {
    const response = await fetch('http://127.0.0.1:8000/api/scan-attendance', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ image_base64: imageSrc }),
      signal: controller.signal
    });
```

```
    clearTimeout(timeoutId);

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const data = await response.json();
    console.log("Scan Response:", data);

    if (data.status === 'success') {
      setScanResult({
        name: data.name,
        code: data.user_code,
        uid: data.uid,
        distance: data.distance
      });
      setVerifiedToast(true);
      setHudType('success');
      setHudStatus(`Verified: ${data.name}`);
      const newLog = {
        id: Date.now(),
        uid: data.uid,
        code: data.user_code,
        name: data.name,
        time: new Date().toLocaleTimeString(),
        status: 'Verified'
      };
      setLogs(prev => [newLog, ...prev]);
      setTimeout(() => {
        setVerifiedToast(false);
        setScanResult(null);
        setHudType('neutral');
        setHudStatus("Ready to Scan");
      }, 2000);
    } else if (data.status === 'already_marked') {
       setScanResult(null);
       setHudStatus(`Already Marked Today — ${data.name}`);
       setHudType('warning');
       setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
    } else if (data.status === 'unknown') {
       setScanResult(null);
       setHudStatus("Access Denied — Unknown Face");
       setHudType('error');
       setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
    } else if (data.status === 'no_face') {
       setScanResult(null);
       setHudStatus("Align Face in Frame");
       setHudType('warning');
       setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
    } else {
       setScanResult(null);
       setHudStatus(data.message || 'Face not recognized');
       setHudType('error');
       setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
    }
  } catch (error) {
    console.error("Fetch Scan Error:", error);
    setScanResult(null);
    setHudStatus("Scan Error / Retry");
    setHudType('error');
    setTimeout(() => { setHudStatus("Ready to Scan"); setHudType('neutral'); }, 2000);
  }
};

const handleEnroll = async (e: React.FormEvent) => {
  e.preventDefault();
  if (!enrollWebcamRef.current) return;
  if (!enrollCode || !enrollName) {
    alert("Please provide Code, Name, and ensure camera is active.");
    return;
  }

  setIsEnrolling(true);
  setEnrollStatus('Capturing 5 face samples... Please look at the camera.');

  const images: string[] = [];
  for (let i = 0; i < 5; i++) {
    const src = enrollWebcamRef.current.getScreenshot();
    if (src) images.push(src);
    setEnrollStatus(`Capturing sample ${i + 1}/5...`);
    await new Promise(r => setTimeout(r, 300));
  }

  setEnrollStatus('Processing biometrics...');

  try {
```

```
    const response = await fetch('http://127.0.0.1:8000/api/enroll', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        user_code: enrollCode,
        name: enrollName,
        images_base64: images
      }),
    });
    const data = await response.json();

    if (response.ok && data.status === 'success') {
      alert(`Successfully enrolled ${data.name}!`);
      setEnrollCode('');
      setEnrollName('');
      setEnrollStatus('');
      // Refresh users
      fetch('http://127.0.0.1:8000/api/users')
        .then(res => res.json())
        .then(resData => setUsers(resData));
    } else {
      alert(`Enrollment failed: ${data.detail || data.message || 'Unknown error'}`);
      setEnrollStatus('Enrollment failed.');
    }
  } catch (error) {
    console.error(error);
    alert('Error connecting to backend');
    setEnrollStatus('');
  }
  setIsEnrolling(false);
};

const handleAdminLogin = async (e: React.FormEvent) => {
  e.preventDefault();
  try {
    const res = await fetch('http://127.0.0.1:8000/api/admin/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ username: adminUser, password: adminPass })
    });
    if (res.ok) {
      setIsAdminAuth(true);
      setShowAdminLogin(false);
      setActiveTab('directory');
      setAdminUser('');
      setAdminPass('');
    } else {
      alert("Invalid Admin Credentials");
    }
  } catch {
    alert("Error connecting to backend");
  }
};

const filteredUsers = users.filter(u =>
  u.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
  u.code.toLowerCase().includes(searchQuery.toLowerCase())
);

const handleEditUser = (user: any) => {
  setEditingUser(user);
  setEditCode(user.code);
  setEditName(user.name);
};

const handleSaveEdit = async () => {
  if (!editingUser) return;
  try {
    const res = await fetch(`http://127.0.0.1:8000/api/users/${editingUser.uid}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ user_code: editCode, name: editName })
    });
    if (res.ok) {
      setEditingUser(null);
      fetch('http://127.0.0.1:8000/api/users')
        .then(res => res.json())
        .then(data => setUsers(data));
    } else {
      const err = await res.json();
      alert(err.detail || "Error updating user");
    }
  } catch (e) {
    alert("Error updating user");
  }
};
```

```
  const handleDeleteUser = async (user: any) => {
    if (!confirm(`Are you sure you want to delete ${user.name} and ALL their attendance records? This cannot be undone.`)) return;
    try {
      const res = await fetch(`http://127.0.0.1:8000/api/users/${user.uid}`, {
        method: 'DELETE'
      });
      if (res.ok) {
        fetch('http://127.0.0.1:8000/api/users')
          .then(res => res.json())
          .then(data => setUsers(data));
      } else {
        const err = await res.json();
        alert(err.detail || "Error deleting user");
      }
    } catch (e) {
      alert("Error deleting user");
    }
  };

  return (
    <div className="min-h-screen bg-[#060913] text-slate-100 flex flex-col font-sans selection:bg-cyan-500/30">

      {/* Glow Backdrops */}
      <div className="fixed top-0 left-1/4 w-96 h-96 bg-cyan-500/10 rounded-full blur-3xl pointer-events-none" />
      <div className="fixed bottom-0 right-1/4 w-96 h-96 bg-blue-600/10 rounded-full blur-3xl pointer-events-none" />

      {/* Top Futuristic Navbar */}
      <header className="h-16 border-b border-slate-800/80 px-8 flex items-center justify-between bg-[#0A0F1D]/80 backdrop-blur-xl
sticky top-0 z-50">
        <div className="flex items-center gap-3">
          <div className="w-10 h-10 rounded-xl bg-gradient-to-tr from-cyan-500 to-blue-600 flex items-center justify-center shadow-lg
shadow-cyan-500/25 border border-cyan-400/30">
            <Camera className="w-5 h-5 text-white" />
          </div>
          <div>
            <div className="flex items-center gap-2">
              <span className="text-base font-bold tracking-wider text-white">AURA<span className="text-cyan-400">FACE</span></span>
              <span className="text-[10px] uppercase font-mono tracking-widest px-2 py-0.5 rounded bg-cyan-950/80 border border-cyan-
500/30 text-cyan-300 font-semibold">
                NEURAL HUD
              </span>
            </div>
            <p className="text-[11px] text-slate-400">Enterprise Biometric Attendance Suite</p>
          </div>
        </div>

        {/* View Switcher Pill */}
        <div className="flex bg-[#0F172A] p-1 rounded-xl border border-slate-800 shadow-inner">
          <button
            onClick={() => setActiveTab('kiosk')}
            className={`px-5 py-2 rounded-lg text-xs font-semibold transition-all duration-200 flex items-center gap-2 ${activeTab
=== 'kiosk'
              ? 'bg-gradient-to-r from-cyan-500 to-blue-600 text-slate-950 shadow-md shadow-cyan-500/30'
              : 'text-slate-400 hover:text-white'
              }`}>
            <Camera className="w-4 h-4" /> Kiosk Scanner
          </button>
          <button
            onClick={() => {
              if (isAdminAuth) {
                setActiveTab('directory');
              } else {
                setShowAdminLogin(true);
              }
            }}
            className={`px-5 py-2 rounded-lg text-xs font-semibold transition-all duration-200 flex items-center gap-2 ${activeTab
=== 'directory'
              ? 'bg-gradient-to-r from-cyan-500 to-blue-600 text-slate-950 shadow-md shadow-cyan-500/30'
              : 'text-slate-400 hover:text-white'
              }`}>
            <Users className="w-4 h-4" /> Directory & Logs
          </button>
        </div>

        {/* Engine Status Badge */}
        <div className="flex items-center gap-3">
          <div className="flex items-center gap-2 text-xs font-medium px-3.5 py-1.5 rounded-full bg-emerald-950/50 border border-
emerald-500/30 text-emerald-400 shadow-sm shadow-emerald-500/10">
            <span className="w-2 h-2 rounded-full bg-emerald-400 animate-ping" />
            Core Models Online
          </div>
        </div>
      </header>

      {/* Main Container */}
```

```
      <main className="flex-1 max-w-7xl w-full mx-auto p-6 md:p-8 flex flex-col gap-6 relative z-10">

        {/* KPI Metrics Strip */}
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-4 gap-4">
          <div className="bg-[#0D1424]/70 border border-slate-800/80 rounded-2xl p-4 flex items-center justify-between backdrop-blur-
md hover:border-cyan-500/40 transition">
            <div>
              <p className="text-xs text-slate-400 uppercase tracking-wider font-medium">Registered Total</p>
              <h3 className="text-2xl font-bold mt-1 text-white">{users.length} Users</h3>
              <p className="text-[11px] text-cyan-400/80 mt-0.5">Total Enrolled</p>
            </div>
            <div className="p-3 bg-cyan-500/10 border border-cyan-500/20 rounded-xl text-cyan-400">
              <Users className="w-5 h-5" />
            </div>
          </div>

          <div className="bg-[#0D1424]/70 border border-slate-800/80 rounded-2xl p-4 flex items-center justify-between backdrop-blur-
md hover:border-emerald-500/40 transition">
            <div>
              <p className="text-xs text-slate-400 uppercase tracking-wider font-medium">Present Today</p>
              <h3 className="text-2xl font-bold mt-1 text-emerald-400">{presentToday} Punches</h3>
              <p className="text-[11px] text-emerald-400/80 mt-0.5">Successful Scans</p>
            </div>
            <div className="p-3 bg-emerald-500/10 border border-emerald-500/20 rounded-xl text-emerald-400">
              <UserCheck className="w-5 h-5" />
            </div>
          </div>

          <div className="bg-[#0D1424]/70 border border-slate-800/80 rounded-2xl p-4 flex items-center justify-between backdrop-blur-
md hover:border-rose-500/40 transition">
            <div>
              <p className="text-xs text-slate-400 uppercase tracking-wider font-medium">Absent Today</p>
              <h3 className="text-2xl font-bold mt-1 text-rose-400">{absentToday} Missing</h3>
              <p className="text-[11px] text-rose-400/80 mt-0.5">Yet to arrive</p>
            </div>
            <div className="p-3 bg-rose-500/10 border border-rose-500/20 rounded-xl text-rose-400">
              <AlertTriangle className="w-5 h-5" />
            </div>
          </div>

          <div className="bg-[#0D1424]/70 border border-slate-800/80 rounded-2xl p-4 flex items-center justify-between backdrop-blur-
md hover:border-indigo-500/40 transition">
            <div>
              <p className="text-xs text-slate-400 uppercase tracking-wider font-medium">Attendance Rate %</p>
              <h3 className="text-2xl font-bold mt-1 text-indigo-300">{attendanceRate}%</h3>
              <p className="text-[11px] text-indigo-400/80 mt-0.5">Daily Turnout</p>
            </div>
            <div className="p-3 bg-indigo-500/10 border border-indigo-500/20 rounded-xl text-indigo-400">
              <Activity className="w-5 h-5" />
            </div>
          </div>
        </div>

        {activeTab === 'kiosk' ? (
          /* ========================================================================== */
          /* KIOSK SCANNER VIEW                                                         */
          /* ========================================================================== */
          <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 flex-1">

            {/* Viewport Scanner Card */}
            <div className="lg:col-span-8 bg-[#0D1424]/60 border border-slate-800 rounded-3xl p-6 flex flex-col justify-between
backdrop-blur-xl relative overflow-hidden shadow-2xl">

              {/* Header inside viewport */}
              <div className="flex items-center justify-between mb-4">
                <div className="flex items-center gap-2">
                  <span className="h-2.5 w-2.5 rounded-full bg-cyan-400 animate-ping"></span>
                  <span className="text-xs font-semibold uppercase tracking-wider text-slate-200">
                    Live Optical Feed • YuNet ONNX
                  </span>
                </div>
                <div className="flex items-center gap-3 font-mono text-xs text-cyan-400/90">
                  <span>RES: 640x480</span>
                  <span>•</span>
                  <span>FPS: 30.0</span>
                </div>
              </div>

              {/* Camera Frame Simulation */}
              <div className="relative w-full aspect-video bg-slate-950/90 rounded-2xl border border-slate-800 flex items-center
justify-center overflow-hidden group shadow-inner">
                {isCameraOn ? (
                  <Webcam
                    ref={webcamRef}
                    audio={false}
                    screenshotFormat="image/jpeg"
```

```
                  videoConstraints={{ width: 640, height: 480, facingMode: "user" }}
                  className="absolute inset-0 w-full h-full object-cover"
                />
              ) : (
                <div className="absolute inset-0 w-full h-full bg-slate-900 flex items-center justify-center text-slate-500 font-
medium z-10">
                  Camera Offline
                </div>
              )}

              {/* Cyber Scanline Overlay */}
              <div className="absolute inset-0 bg-[linear-gradient(to_bottom,transparent_50%,rgba(6,182,212,0.06)_51%)] bg-
[length:100%_4px] pointer-events-none" />

              {/* Vertical Scanning Beam Animation */}
              <div className="absolute inset-0 bg-gradient-to-b from-transparent via-cyan-500/15 to-transparent h-24 w-full
animate-bounce pointer-events-none opacity-70" />

              {/* Biometric Target Bracket HUD */}
              <div className={`relative w-72 h-72 border-2 rounded-3xl flex flex-col justify-between p-4 transition-colors
duration-300 ${
                hudType === 'success' ? 'border-emerald-500/80 shadow-[0_0_50px_rgba(16,185,129,0.2)]' :
                hudType === 'warning' ? 'border-amber-500/80 shadow-[0_0_50px_rgba(245,158,11,0.2)]' :
                hudType === 'error' ? 'border-rose-500/80 shadow-[0_0_50px_rgba(244,63,94,0.2)]' :
                'border-cyan-400/30 shadow-[0_0_50px_rgba(6,182,212,0.15)]'
              }`}>
                {/* Glowing Reticle Corners */}
                <div className={`absolute -top-1 -left-1 w-6 h-6 border-t-2 border-l-2 rounded-tl-lg transition-colors duration-300
${
                  hudType === 'success' ? 'border-emerald-400' : hudType === 'warning' ? 'border-amber-500' : hudType === 'error' ?
'border-rose-500' : 'border-cyan-400'
                }`} />
                <div className={`absolute -top-1 -right-1 w-6 h-6 border-t-2 border-r-2 rounded-tr-lg transition-colors duration-
300 ${
                  hudType === 'success' ? 'border-emerald-400' : hudType === 'warning' ? 'border-amber-500' : hudType === 'error' ?
'border-rose-500' : 'border-cyan-400'
                }`} />
                <div className={`absolute -bottom-1 -left-1 w-6 h-6 border-b-2 border-l-2 rounded-bl-lg transition-colors duration-
300 ${
                  hudType === 'success' ? 'border-emerald-400' : hudType === 'warning' ? 'border-amber-500' : hudType === 'error' ?
'border-rose-500' : 'border-cyan-400'
                }`} />
                <div className={`absolute -bottom-1 -right-1 w-6 h-6 border-b-2 border-r-2 rounded-br-lg transition-colors
duration-300 ${
                  hudType === 'success' ? 'border-emerald-400' : hudType === 'warning' ? 'border-amber-500' : hudType === 'error' ?
'border-rose-500' : 'border-cyan-400'
                }`} />

                {/* Top HUD Tag */}
                <div className="flex justify-between items-center text-[10px] font-mono text-cyan-300 bg-cyan-950/80 px-2.5 py-1
rounded-md border border-cyan-500/30 backdrop-blur-md">
                  <span>{scanResult ? `MATCH: ${(100 - scanResult.distance*100).toFixed(1)}%` : "AWAITING FACE"}</span>
                  <span>{scanResult ? `DIST: ${scanResult.distance}` : "PRESS SCAN"}</span>
                </div>

                {/* Identified Subject Badge */}
                <div className="text-center w-full flex justify-center">
                    {scanResult ? (
                  <span className="text-xs font-semibold px-4 py-1.5 bg-emerald-500/20 text-emerald-300 border border-emerald-
500/40 rounded-full backdrop-blur-md shadow-lg shadow-emerald-500/10 whitespace-nowrap">
                    {hudStatus}
                  </span>
                  ) : (
                  <span className={`text-xs font-semibold px-4 py-1.5 rounded-full backdrop-blur-md shadow-lg ${
                    hudType === 'error'
                      ? 'bg-rose-500/20 text-rose-300 border border-rose-500/40'
                      : hudType === 'warning'
                      ? 'bg-amber-500/20 text-amber-300 border border-amber-500/40'
                      : 'bg-slate-800/80 text-slate-400 border border-slate-700'
                  }`}>
                    {hudStatus}
                  </span>
                  )}
                </div>
              </div>

              {/* Bottom Liveness Prompt Bar (Removed) */}

              {/* Toast Notification on Successful Verification */}
              {verifiedToast && scanResult && (
                <div className="absolute top-6 inset-x-12 bg-emerald-950/90 border border-emerald-500/50 p-4 rounded-xl flex items-
center justify-between text-emerald-200 backdrop-blur-2xl shadow-2xl animate-in fade-in slide-in-from-top duration-300 z-30">
                  <div className="flex items-center gap-3">
                    <CheckCircle2 className="w-6 h-6 text-emerald-400" />
                    <div>
                      <h4 className="text-sm font-bold">Attendance Verified!</h4>
```

```
                      <p className="text-xs text-emerald-300/80">Welcome, {scanResult.name}. Punch saved to SQLite & logs.</p>
                    </div>
                  </div>
                  <span className="text-xs font-mono bg-emerald-900/60 px-2 py-1 rounded border border-emerald-500/30">
                    UID #{scanResult.uid} • {scanResult.distance} L2
                  </span>
                </div>
              )}
            </div>

            {/* Viewport Bottom Controls */}
            <div className="mt-5 flex items-center justify-between">
              <button
                onClick={toggleCamera}
                className={`px-4 py-2.5 rounded-xl text-xs font-bold transition-all shadow-md ${isCameraOn ? 'bg-slate-800 text-slate-300 hover:bg-slate-700' : 'bg-rose-500/20 text-rose-400 hover:bg-rose-500/30'} border border-slate-700`}>
                {isCameraOn ? "Camera OFF" : "Camera ON"}
              </button>
              <button
                onClick={handleLiveScan}
                className="px-6 py-2.5 rounded-xl bg-gradient-to-r from-cyan-500 to-blue-600 hover:from-cyan-400 hover:to-blue-500 text-slate-950 text-xs font-bold transition-all shadow-lg shadow-cyan-500/25 active:scale-95">
                Scan & Mark Attendance
              </button>
            </div>
          </div>

          {/* Right Feed Panel */}
          <div className="lg:col-span-4 bg-[#0D1424]/60 border border-slate-800 rounded-3xl p-6 flex flex-col backdrop-blur-xl shadow-2xl">
            <div className="flex items-center justify-between pb-4 border-b border-slate-800">
              <h3 className="text-sm font-bold text-white flex items-center gap-2">
                <Clock className="w-4 h-4 text-cyan-400" /> Realtime Check-ins
              </h3>
              <span className="text-[11px] font-mono text-cyan-400 bg-cyan-950/60 border border-cyan-500/30 px-2 py-0.5 rounded">
                Live Stream
              </span>
            </div>

            {/* Feed List */}
            <div className="mt-4 flex-1 flex flex-col gap-3 overflow-y-auto max-h-[440px] pr-1">
              {logs.map((log) => (
                <div
                  key={log.id}
                  className="p-3.5 rounded-xl bg-[#0A0F1D]/80 border border-slate-800/80 flex items-center justify-between hover:border-cyan-500/30 transition">
                  <div className="flex items-center gap-3">
                    <div className="w-10 h-10 rounded-xl bg-gradient-to-tr from-slate-800 to-slate-700 border border-slate-700 flex items-center justify-center font-bold text-sm text-cyan-400">
                      {log.name.charAt(0)}
                    </div>
                    <div>
                      <h5 className="text-xs font-bold text-white">{log.name}</h5>
                      <p className="text-[10px] font-mono text-slate-400">ID: {log.code} • UID: #{log.uid}</p>
                    </div>
                  </div>
                  <div className="text-right">
                    <span className="text-xs font-mono text-emerald-400 font-semibold block">{log.time}</span>
                  </div>
                </div>
              ))}
            </div>

            {/* Bottom Quick Action Box */}
            <div className="mt-4 p-4 rounded-2xl bg-gradient-to-b from-cyan-950/30 to-slate-900/60 border border-cyan-500/20 text-center">
              <p className="text-xs font-semibold text-slate-200">Face Not Enrolled?</p>
              <p className="text-[11px] text-slate-400 mt-0.5 mb-3">Register new face samples in the management directory.</p>
              <button
                onClick={() => setActiveTab('directory')}
                className="w-full py-2.5 bg-slate-800 hover:bg-slate-700 text-cyan-300 border border-cyan-500/30 text-xs font-semibold rounded-xl transition flex items-center justify-center gap-2">
                <UserPlus className="w-3.5 h-3.5" /> Go to User Directory
              </button>
            </div>
          </div>

        </div>
      ) : (
        /* ========================================================================== */
        /* USER DIRECTORY & MANAGEMENT VIEW                                           */
        /* ========================================================================== */
        <div className="grid grid-cols-1 lg:grid-cols-12 gap-6 flex-1">

          {/* Left: Enrollment Form */}
          <div className="lg:col-span-4 bg-[#0D1424]/60 border border-slate-800 rounded-3xl p-6 backdrop-blur-xl flex flex-col
```

```
justify-between shadow-2xl">
              <div>
                <div className="flex items-center gap-2 mb-1">
                  <UserPlus className="w-4 h-4 text-cyan-400" />
                  <h3 className="text-sm font-bold text-white">Biometric Enrollment</h3>
                </div>
                <p className="text-[11px] text-slate-400 mb-6">Extracts 5-vector temporal samples into SQLite & FAISS</p>

                <form className="space-y-4" onSubmit={handleEnroll}>
                  <div>
                    <label className="text-xs font-medium text-slate-300 block mb-1.5">User / Employee Code</label>
                    <input
                      type="text"
                      value={enrollCode}
                      onChange={(e) => setEnrollCode(e.target.value)}
                      placeholder="e.g. EMP106"
                      className="w-full bg-[#0A0F1D] border border-slate-700/80 rounded-xl px-4 py-2.5 text-xs text-white
placeholder-slate-500 focus:border-cyan-500 focus:outline-none transition"
                      />
                  </div>

                  <div>
                    <label className="text-xs font-medium text-slate-300 block mb-1.5">Full Name</label>
                    <input
                      type="text"
                      value={enrollName}
                      onChange={(e) => setEnrollName(e.target.value)}
                      placeholder="e.g. Rohini Rathore"
                      className="w-full bg-[#0A0F1D] border border-slate-700/80 rounded-xl px-4 py-2.5 text-xs text-white
placeholder-slate-500 focus:border-cyan-500 focus:outline-none transition"
                      />
                  </div>

                  <div>
                    <label className="text-xs font-medium text-slate-300 block mb-1.5">Biometric Sensor Viewport</label>
                    <div className="w-full h-40 rounded-xl bg-slate-950 border border-slate-800 flex flex-col items-center justify-
center text-slate-500 gap-2 relative overflow-hidden">
                      <Webcam
                        ref={enrollWebcamRef}
                        audio={false}
                        screenshotFormat="image/jpeg"
                        videoConstraints={{ width: 320, height: 240, facingMode: "user" }}
                        className="absolute inset-0 w-full h-full object-cover opacity-80"
                      />
                      <div className="absolute inset-x-0 bottom-0 h-1 bg-cyan-500/30" />
                    </div>
                  </div>

                  <button
                    type="submit"
                    disabled={isEnrolling}
                    className="w-full py-3 rounded-xl bg-gradient-to-r from-emerald-500 to-teal-600 hover:from-emerald-400 hover:to-
teal-500 text-slate-950 font-bold text-xs shadow-lg shadow-emerald-500/20 transition active:scale-95 disabled:opacity-50">
                    {isEnrolling ? 'Enrolling...' : 'Enroll Biometrics (Strict 0.15 Gated)'}
                  </button>
                  {enrollStatus && <p className="text-xs text-emerald-400 text-center mt-2">{enrollStatus}</p>}
                </form>
              </div>

              <div className="mt-6 pt-4 border-t border-slate-800/80 text-[11px] text-slate-400 flex items-center gap-2">
                <ShieldCheck className="w-4 h-4 text-emerald-400 shrink-0" />
                Duplicate prevention guarantees zero duplicate registrations.
              </div>
            </div>

            {/* Right: Registered Database Table */}
            <div className="lg:col-span-8 bg-[#0D1424]/60 border border-slate-800 rounded-3xl p-6 backdrop-blur-xl flex flex-col
shadow-2xl">

              {/* Table Toolbar */}
              <div className="flex flex-col sm:flex-row items-center justify-between gap-4 pb-5 border-b border-slate-800">
                <div>
                  <div className="flex items-center gap-4 mb-2">
                    <button onClick={() => setReportTab('directory')} className={`text-sm font-bold pb-1 transition-colors
${reportTab === 'directory' ? 'text-white border-b-2 border-cyan-500' : 'text-slate-400 hover:text-white'}`}>Directory</button>
                    <button onClick={() => setReportTab('daily')} className={`text-sm font-bold pb-1 transition-colors ${reportTab
=== 'daily' ? 'text-white border-b-2 border-cyan-500' : 'text-slate-400 hover:text-white'}`}>Daily Report</button>
                    <button onClick={() => setReportTab('monthly')} className={`text-sm font-bold pb-1 transition-colors ${reportTab
=== 'monthly' ? 'text-white border-b-2 border-cyan-500' : 'text-slate-400 hover:text-white'}`}>Monthly Report</button>
                  </div>
                  <p className="text-[11px] text-slate-400">
                    {reportTab === 'directory' ? `Total ${users.length} active identities indexed in FAISS` :
                     reportTab === 'daily' ? `Daily attendance insights` : `Monthly summary insights`}
                  </p>
                </div>
```

```
              <div className="flex items-center gap-3 w-full sm:w-auto">
                {reportTab === 'directory' && (
                  <>
                    <div className="relative flex-1 sm:w-60">
                      <Search className="w-3.5 h-3.5 absolute left-3.5 top-1/2 -translate-y-1/2 text-slate-400" />
                      <input
                        type="text"
                        placeholder="Search code or name..."
                        value={searchQuery}
                        onChange={(e) => setSearchQuery(e.target.value)}
                        className="w-full bg-[#0A0F1D] border border-slate-700/80 rounded-xl pl-9 pr-3 py-2 text-xs text-white
placeholder-slate-500 focus:border-cyan-500 focus:outline-none transition"
                      />
                    </div>
                    <button
                      onClick={() => window.open('http://127.0.0.1:8000/api/export/today', '_blank')}
                      className="px-3 py-2 bg-slate-800 hover:bg-slate-700 text-slate-300 rounded-xl text-xs font-semibold flex
items-center gap-1.5 transition">
                      <Download className="w-3.5 h-3.5" /> Export Today
                    </button>
                    <button
                      onClick={() => window.open('http://127.0.0.1:8000/api/export/summary', '_blank')}
                      className="px-3 py-2 bg-slate-800 hover:bg-slate-700 text-slate-300 rounded-xl text-xs font-semibold flex
items-center gap-1.5 transition">
                      <Download className="w-3.5 h-3.5" /> Summary
                    </button>
                  </>
                )}
                {reportTab === 'daily' && (
                  <>
                    <input
                      type="date"
                      value={dailyDate}
                      onChange={(e) => setDailyDate(e.target.value)}
                      className="bg-[#0A0F1D] border border-slate-700/80 rounded-xl px-3 py-2 text-xs text-white focus:border-cyan-
500 focus:outline-none transition"
                    />
                    <button
                      onClick={() => {
                         const csvRows = [["User Code", "Name", "Date", "Punch Time", "Status"]];
                         dailyReport?.records?.forEach((r: any) => csvRows.push([r.user_code, r.name, r.date, r.punch_time,
r.status]));

                         const csvString = csvRows.map((r: any) => r.join(",")).join("\n");
                         const blob = new Blob([csvString], { type: 'text/csv' });
                         const url = window.URL.createObjectURL(blob);
                         const a = document.createElement('a');
                         a.href = url;
                         a.download = `Daily_Report_${dailyDate}.csv`;
                         a.click();
                      }}
                      className="px-3 py-2 bg-slate-800 hover:bg-slate-700 text-slate-300 rounded-xl text-xs font-semibold flex
items-center gap-1.5 transition">
                      <Download className="w-3.5 h-3.5" /> Export CSV
                    </button>
                  </>
                )}
                {reportTab === 'monthly' && (
                  <>
                    <input
                      type="month"
                      value={monthlyMonth}
                      onChange={(e) => setMonthlyMonth(e.target.value)}
                      className="bg-[#0A0F1D] border border-slate-700/80 rounded-xl px-3 py-2 text-xs text-white focus:border-cyan-
500 focus:outline-none transition"
                    />
                    <button
                      onClick={() => {
                         const csvRows = [["User Code", "Name", "Month", "Total Days", "Days Present", "Days Absent", "Attendance
%"]];

                         monthlyReport?.records?.forEach((r: any) => csvRows.push([r.user_code, r.name, r.month, r.total_days,
r.days_present, r.days_absent, r.attendance_percentage]));
                         const csvString = csvRows.map((r: any) => r.join(",")).join("\n");
                         const blob = new Blob([csvString], { type: 'text/csv' });
                         const url = window.URL.createObjectURL(blob);
                         const a = document.createElement('a');
                         a.href = url;
                         a.download = `Monthly_Report_${monthlyMonth}.csv`;
                         a.click();
                      }}
                      className="px-3 py-2 bg-slate-800 hover:bg-slate-700 text-slate-300 rounded-xl text-xs font-semibold flex
items-center gap-1.5 transition">
                      <Download className="w-3.5 h-3.5" /> Export CSV
                    </button>
                  </>
                )}
              </div>
```

```
            </div>

            {/* Data Table */}
            <div className="mt-4 flex-1 overflow-x-auto">
              {reportTab === 'directory' && (
                <table className="w-full text-left border-collapse">
                  <thead>
                    <tr className="border-b border-slate-800 text-[11px] font-mono uppercase text-slate-400">
                      <th className="py-3 px-4">S.No</th>
                      <th className="py-3 px-4">UID</th>
                      <th className="py-3 px-4">User Code</th>
                      <th className="py-3 px-4">Full Name</th>
                      <th className="py-3 px-4">Reg Date</th>
                      <th className="py-3 px-4 text-right">Actions</th>
                    </tr>
                  </thead>
                  <tbody className="divide-y divide-slate-800/60 text-xs">
                    {filteredUsers.map((user, idx) => (
                      <tr key={user.uid} className="hover:bg-slate-800/30 transition group">
                        <td className="py-3.5 px-4 font-mono text-slate-400">{idx + 1}</td>
                        <td className="py-3.5 px-4 font-mono font-bold text-cyan-400">#{user.uid}</td>
                        <td className="py-3.5 px-4 font-mono text-slate-300">{user.code}</td>
                        <td className="py-3.5 px-4 font-medium text-white">{user.name}</td>
                        <td className="py-3.5 px-4 text-slate-400 font-mono text-[11px]">{user.date}</td>
                        <td className="py-3.5 px-4 text-right">
                          <div className="flex items-center justify-end gap-2">
                            <button onClick={() => setIndividualReportUser(user)} className="p-1.5 hover:bg-slate-700 text-slate-
400 hover:text-cyan-400 rounded-lg transition" title="View Report">
                              <Activity className="w-3.5 h-3.5" />
                            </button>
                            <button onClick={() => handleEditUser(user)} className="p-1.5 hover:bg-slate-700 text-slate-400
hover:text-cyan-400 rounded-lg transition" title="Edit Record">
                              <Edit3 className="w-3.5 h-3.5" />
                            </button>
                            <button onClick={() => handleDeleteUser(user)} className="p-1.5 hover:bg-rose-500/20 text-slate-400
hover:text-rose-400 rounded-lg transition" title="Delete User">
                              <Trash2 className="w-3.5 h-3.5" />
                            </button>
                          </div>
                        </td>
                      </tr>
                    ))}
                  </tbody>
                </table>
              )}

              {reportTab === 'daily' && dailyReport && (
                <div>
                  <div className="grid grid-cols-4 gap-4 mb-4">
                    <div className="bg-[#0A0F1D] p-3 rounded-xl border border-slate-800/80 text-center">
                      <div className="text-[10px] text-slate-400 uppercase">Total Users</div>
                      <div className="text-lg font-bold text-white">{dailyReport.total_users}</div>
                    </div>
                    <div className="bg-emerald-900/20 p-3 rounded-xl border border-emerald-500/20 text-center">
                      <div className="text-[10px] text-emerald-400 uppercase">Present</div>
                      <div className="text-lg font-bold text-emerald-400">{dailyReport.present_count}</div>
                    </div>
                    <div className="bg-rose-900/20 p-3 rounded-xl border border-rose-500/20 text-center">
                      <div className="text-[10px] text-rose-400 uppercase">Absent</div>
                      <div className="text-lg font-bold text-rose-400">{dailyReport.absent_count}</div>
                    </div>
                    <div className="bg-cyan-900/20 p-3 rounded-xl border border-cyan-500/20 text-center">
                      <div className="text-[10px] text-cyan-400 uppercase">Attendance Rate</div>
                      <div className="text-lg font-bold text-cyan-400">{dailyReport.attendance_rate}%</div>
                    </div>
                  </div>
                  <table className="w-full text-left border-collapse">
                    <thead>
                      <tr className="border-b border-slate-800 text-[11px] font-mono uppercase text-slate-400">
                        <th className="py-3 px-4">User Code</th>
                        <th className="py-3 px-4">Name</th>
                        <th className="py-3 px-4">Date</th>
                        <th className="py-3 px-4">Check-in Time</th>
                        <th className="py-3 px-4">Status</th>
                      </tr>
                    </thead>
                    <tbody className="divide-y divide-slate-800/60 text-xs">
                      {dailyReport.records.map((r: any) => (
                        <tr key={r.uid} className="hover:bg-slate-800/30 transition">
                          <td className="py-3 px-4 font-mono text-slate-300">{r.user_code}</td>
                          <td className="py-3 px-4 font-medium text-white">{r.name}</td>
                          <td className="py-3 px-4 text-slate-400">{r.date}</td>
                          <td className="py-3 px-4 font-mono text-cyan-400">{r.punch_time}</td>
                          <td className="py-3 px-4">
                            {r.status === 'Present' ? (
                              <span className="px-2 py-1 bg-emerald-500/20 text-emerald-400 rounded-md text-[10px] font-bold border
```

```
border-emerald-500/30">PRESENT</span>
                            ) : (
                              <span className="px-2 py-1 bg-rose-500/20 text-rose-400 rounded-md text-[10px] font-bold border
border-rose-500/30">ABSENT</span>
                            )}
                          </td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
              )}

              {reportTab === 'monthly' && monthlyReport && (
                <div>
                  <div className="mb-4 bg-[#0A0F1D] p-3 rounded-xl border border-slate-800/80 text-center inline-block">
                      <div className="text-[10px] text-slate-400 uppercase px-4">Working Days</div>
                      <div className="text-lg font-bold text-white px-4">{monthlyReport.working_days}</div>
                  </div>
                  <table className="w-full text-left border-collapse">
                    <thead>
                      <tr className="border-b border-slate-800 text-[11px] font-mono uppercase text-slate-400">
                        <th className="py-3 px-4">User Code</th>
                        <th className="py-3 px-4">Name</th>
                        <th className="py-3 px-4 text-center">Total Days</th>
                        <th className="py-3 px-4 text-center">Days Present</th>
                        <th className="py-3 px-4 text-center">Days Absent</th>
                        <th className="py-3 px-4 text-right">Attendance Rate</th>
                      </tr>
                    </thead>
                    <tbody className="divide-y divide-slate-800/60 text-xs">
                      {monthlyReport.records.map((r: any) => (
                        <tr key={r.uid} className="hover:bg-slate-800/30 transition">
                          <td className="py-3 px-4 font-mono text-slate-300">{r.user_code}</td>
                          <td className="py-3 px-4 font-medium text-white">{r.name}</td>
                          <td className="py-3 px-4 text-center text-slate-400 font-mono">{r.total_days}</td>
                          <td className="py-3 px-4 text-center text-emerald-400 font-mono">{r.days_present}</td>
                          <td className="py-3 px-4 text-center text-rose-400 font-mono">{r.days_absent}</td>
                          <td className="py-3 px-4 text-right text-cyan-400 font-bold">{r.attendance_percentage}%</td>
                        </tr>
                      ))}
                    </tbody>
                  </table>
                </div>
              )}
            </div>

            {/* Sequential Integrity Notice */}
            <div className="mt-4 pt-3 border-t border-slate-800 flex items-center justify-between text-[11px] text-slate-400">
              <span>Vector Dimension: 128-d Unit Normal L2</span>
              <span className="font-mono text-cyan-400">FAISS Status: Synchronized</span>
            </div>
          </div>

        </div>
      )}

    </main>

    {/* Admin Login Modal */}
    {showAdminLogin && (
      <div className="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-[100]">
        <div className="bg-[#0D1424] border border-slate-700 p-8 rounded-2xl w-full max-w-md shadow-2xl relative">
          <button onClick={() => setShowAdminLogin(false)} className="absolute top-4 right-4 text-slate-400 hover:text-
white">✕</button>
          <h3 className="text-xl font-bold text-white flex items-center gap-2 mb-6">
            <ShieldCheck className="w-6 h-6 text-cyan-400" /> Admin Portal Login
          </h3>
          <form onSubmit={handleAdminLogin} className="space-y-4">
            <div>
              <label className="text-xs text-slate-400 mb-1 block">Username</label>
              <input type="text" value={adminUser} onChange={e => setAdminUser(e.target.value)} className="w-full bg-[#0A0F1D]
border border-slate-700 rounded-xl px-4 py-2.5 text-sm text-white" />
            </div>
            <div>
              <label className="text-xs text-slate-400 mb-1 block">Password</label>
              <input type="password" value={adminPass} onChange={e => setAdminPass(e.target.value)} className="w-full bg-[#0A0F1D]
border border-slate-700 rounded-xl px-4 py-2.5 text-sm text-white" />
            </div>
            <button type="submit" className="w-full py-3 bg-blue-600 hover:bg-blue-500 text-white font-bold rounded-xl mt-
4">Login</button>
          </form>
        </div>
      </div>
    )}
```

```
      {/* Edit User Modal */}
      {editingUser && (
        <div className="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-[100]">
          <div className="bg-[#0D1424] border border-slate-700 p-8 rounded-2xl w-full max-w-md shadow-2xl relative">
            <button onClick={() => setEditingUser(null)} className="absolute top-4 right-4 text-slate-400 hover:text-
white">✕</button>
            <h3 className="text-xl font-bold text-white flex items-center gap-2 mb-6">
              <Edit3 className="w-6 h-6 text-cyan-400" /> Edit User Record
            </h3>
            <div className="space-y-4">
              <div>
                <label className="text-xs text-slate-400 mb-1 block">User Code</label>
                <input type="text" value={editCode} onChange={e => setEditCode(e.target.value)} className="w-full bg-[#0A0F1D] border
border-slate-700 rounded-xl px-4 py-2.5 text-sm text-white" />
              </div>
              <div>
                <label className="text-xs text-slate-400 mb-1 block">Name</label>
                <input type="text" value={editName} onChange={e => setEditName(e.target.value)} className="w-full bg-[#0A0F1D] border
border-slate-700 rounded-xl px-4 py-2.5 text-sm text-white" />
              </div>
              <button onClick={handleSaveEdit} className="w-full py-3 bg-cyan-600 hover:bg-cyan-500 text-white font-bold rounded-xl
mt-4">Save Changes</button>
            </div>
          </div>
        </div>
      )}

      {/* Individual User Report Modal */}
      {individualReportUser && (
        <div className="fixed inset-0 bg-black/60 backdrop-blur-sm flex items-center justify-center z-[100]">
          <div className="bg-[#0D1424] border border-slate-700 p-6 rounded-2xl w-full max-w-3xl shadow-2xl relative flex flex-col
max-h-[90vh]">
            <button onClick={() => {setIndividualReportUser(null); setIndividualReport(null);}} className="absolute top-4 right-4
text-slate-400 hover:text-white">✕</button>
            <h3 className="text-xl font-bold text-white flex items-center gap-2 mb-2">
              <Activity className="w-6 h-6 text-cyan-400" /> Individual Attendance Report
            </h3>
            <p className="text-xs text-slate-400 mb-6 font-mono">UID: #{individualReportUser.uid} • {individualReportUser.name}
({individualReportUser.code})</p>

            <div className="flex items-center gap-4 mb-6">
               <label className="text-sm font-semibold text-slate-300">Select Month:</label>
               <input
                 type="month"
                 value={individualReportMonth}
                 onChange={(e) => setIndividualReportMonth(e.target.value)}
                 className="bg-[#0A0F1D] border border-slate-700/80 rounded-xl px-3 py-2 text-sm text-white focus:border-cyan-500
focus:outline-none transition"
               />
            </div>

            {individualReport ? (
               individualReport.status === 'success' ? (
                 <>
                   <div className="grid grid-cols-4 gap-4 mb-6">
                     <div className="bg-[#0A0F1D] p-3 rounded-xl border border-slate-800/80 text-center">
                       <div className="text-[10px] text-slate-400 uppercase">Working Days</div>
                       <div className="text-lg font-bold text-white">{individualReport.total_working_days}</div>
                     </div>
                     <div className="bg-emerald-900/20 p-3 rounded-xl border border-emerald-500/20 text-center">
                       <div className="text-[10px] text-emerald-400 uppercase">Days Present</div>
                       <div className="text-lg font-bold text-emerald-400">{individualReport.days_present}</div>
                     </div>
                     <div className="bg-rose-900/20 p-3 rounded-xl border border-rose-500/20 text-center">
                       <div className="text-[10px] text-rose-400 uppercase">Days Absent</div>
                       <div className="text-lg font-bold text-rose-400">{individualReport.days_absent}</div>
                     </div>
                     <div className="bg-cyan-900/20 p-3 rounded-xl border border-cyan-500/20 text-center">
                       <div className="text-[10px] text-cyan-400 uppercase">Attendance Rate</div>
                       <div className="text-lg font-bold text-cyan-400">{individualReport.attendance_rate}%</div>
                     </div>
                   </div>

                   <h4 className="text-sm font-bold text-white mb-3">Daily Breakdown</h4>
                   <div className="flex-1 overflow-y-auto">
                     <table className="w-full text-left border-collapse">
                       <thead>
                         <tr className="border-b border-slate-800 text-[11px] font-mono uppercase text-slate-400 sticky top-0 bg-
[#0D1424]">
                           <th className="py-2 px-4">Date</th>
                           <th className="py-2 px-4">Punch Time</th>
                           <th className="py-2 px-4">Status</th>
                         </tr>
                       </thead>
                       <tbody className="divide-y divide-slate-800/60 text-xs">
                         {individualReport.daily_breakdown.length > 0 ? individualReport.daily_breakdown.map((r: any, i: number) => (
```

```
                          <tr key={i} className="hover:bg-slate-800/30 transition">
                            <td className="py-2 px-4 text-slate-300 font-mono">{r.date}</td>
                            <td className="py-2 px-4 font-mono text-cyan-400">{r.punch_time}</td>
                            <td className="py-2 px-4">
                              <span className="px-2 py-1 bg-emerald-500/20 text-emerald-400 rounded-md text-[10px] font-bold border
border-emerald-500/30">PRESENT</span>
                            </td>
                          </tr>
                        )) : (
                          <tr><td colSpan={3} className="py-4 text-center text-slate-500">No attendance records for this month.</td>
</tr>
                        )}
                      </tbody>
                    </table>
                  </div>
                </>
              ) : (
                <div className="text-rose-400 p-4 bg-rose-900/20 rounded-xl border border-rose-500/30 text-sm">
                   {individualReport.message || "Error loading report"}
                </div>
              )
            ) : (
               <div className="flex-1 flex items-center justify-center text-slate-400">Loading...</div>
            )}
          </div>
        </div>
      )}
    </div>
  );
}
```

Explanation:

- **State Management**: Uses extensive `useState` hooks to track Kiosk/Directory tabs, Live Logs (`logs`), Registered Users (`users`), active scanning results (`scanResult`), and HUD status visuals (`hudType`, `hudStatus`).
- **Webcam Feed**: Utilizes `react-webcam` for consistent 30fps optical feed capturing. `getScreenshot()` dumps JPEG base64 payloads to the backend.
- **`useEffect` Lifecycle**:
    - On mount: Fetches initial users and today's attendance logs.
    - On dependency change: Refetches daily/monthly reports when the user modifies date/month pickers or tab selection.
- **Live Scanning (`handleLiveScan`)**: Sets an `AbortController` timeout (5s). Posts frame to `/api/scan-attendance`. Evaluates backend status (`success`, `already_marked`, `unknown`, `no_face`) to dynamically update the HUD Reticle styling and trigger 2-second timeout badges without clearing persistent logs.
- **User Management**: `handleEditUser` and `handleDeleteUser` interface with the backend PUT/DELETE routes and instantly refresh the local users state upon success.
- **Reporting Modals**: Client-side rendering of structured tables mapped from backend JSON. Client-side CSV generation constructs Blob URLs for direct downloading.

`app.py`

Standalone Python Tkinter (CustomTkinter) Desktop Client.

```python
import customtkinter as ctk
import cv2
import threading
import time
import csv
from tkinter import messagebox, filedialog, ttk
from PIL import Image
import numpy as np

# Import our backend modules
import db_manager as db
from vector_index import VectorIndex
from face_engine import FaceEngine

class BiometricDashboard(ctk.CTk):
    def __init__(self):
        super().__init__()

        # 1. Palette & Modern Dark Aesthetic
        self.title("AURA-FACE | Biometric Kiosk & Management")
        self.geometry("1240x780")
        self.minsize(1240, 780)

        ctk.set_appearance_mode("Dark")
        # Define custom colors
        self.colors = {
            "bg_deep": "#0B0F19",
            "surface": "#151D2E",
            "border": "#22304A",
            "green": "#10B981",
            "blue": "#3B82F6",
            "red": "#EF4444",
            "purple": "#8B5CF6",
            "text_main": "#F8FAFC",
```

```
            "text_muted": "#94A3B8"
        }
        self.configure(fg_color=self.colors["bg_deep"])

        # Application State
        self.mode = "KIOSK"
        self.cap = None
        self.camera_running = False
        self.enrollment_mode = False
        self.enrollment_crops = []
        self.enrollment_target = 5
        self.enrollment_user_id = None

        # Initialize Backend Engines
        try:
            self.face_engine = FaceEngine()
            self.vector_index = VectorIndex()
        except Exception as e:
            messagebox.showerror("Engine Initialization Error", f"Failed to load AI models:\n{e}")

        # UI Setup
        self.setup_header()
        self.setup_kiosk_view()
        self.update_clock()

        self.protocol("WM_DELETE_WINDOW", self.on_closing)

    def setup_header(self):
        self.header_frame = ctk.CTkFrame(self, fg_color=self.colors["surface"], corner_radius=0, height=60)
        self.header_frame.pack(fill="x", side="top")

        # Branding
        title = ctk.CTkLabel(self.header_frame, text="AURA-FACE | Biometric Kiosk & Management",
                             font=ctk.CTkFont(size=20, weight="bold"), text_color=self.colors["text_main"])
        title.pack(side="left", padx=20, pady=15)

        # Controls (Right side)
        self.admin_btn = ctk.CTkButton(self.header_frame, text="Admin Portal", fg_color="transparent",
                                       border_width=2, border_color=self.colors["blue"], text_color=self.colors["blue"],
                                       hover_color="#1E3A8A", command=self.open_admin_auth)
        self.admin_btn.pack(side="right", padx=20, pady=15)

        self.clock_label = ctk.CTkLabel(self.header_frame, text="00:00:00", font=ctk.CTkFont(family="Consolas", size=18),
text_color=self.colors["text_muted"])
        self.clock_label.pack(side="right", padx=20, pady=15)

    def update_clock(self):
        current_time = time.strftime("%H:%M:%S")
        self.clock_label.configure(text=current_time)
        self.after(1000, self.update_clock)

    def setup_kiosk_view(self):
        self.main_container = ctk.CTkFrame(self, fg_color="transparent")
        self.main_container.pack(fill="both", expand=True, padx=20, pady=20)

        # --- Left Card (Camera Viewport) ---
        self.left_card = ctk.CTkFrame(self.main_container, fg_color=self.colors["surface"], border_width=1,
border_color=self.colors["border"])
        self.left_card.pack(side="left", fill="both", expand=True, padx=(0, 10))

        cam_header = ctk.CTkFrame(self.left_card, fg_color="transparent")
        cam_header.pack(fill="x", padx=20, pady=15)
        ctk.CTkLabel(cam_header, text="Live Scanner Terminal", font=ctk.CTkFont(size=18, weight="bold")).pack(side="left")
        self.cam_badge = ctk.CTkLabel(cam_header, text="● STANDBY", text_color=self.colors["text_muted"],
font=ctk.CTkFont(weight="bold"))
        self.cam_badge.pack(side="right")

        self.video_label = ctk.CTkLabel(self.left_card, text="Camera is Idle.\nClick 'Start Attendance Scan' to begin",
                                        font=ctk.CTkFont(size=16), text_color=self.colors["text_muted"])
        self.video_label.pack(fill="both", expand=True, padx=20, pady=10)

        cam_controls = ctk.CTkFrame(self.left_card, fg_color="transparent")
        cam_controls.pack(fill="x", padx=20, pady=20)

        self.start_scan_btn = ctk.CTkButton(cam_controls, text="▶ Start Attendance Scan", fg_color=self.colors["green"],
                                            hover_color="#059669", height=40, font=ctk.CTkFont(weight="bold"),
command=self.start_kiosk_camera)
        self.start_scan_btn.pack(side="left", expand=True, fill="x", padx=(0, 10))

        self.stop_scan_btn = ctk.CTkButton(cam_controls, text="■ Stop", fg_color=self.colors["red"],
                                           hover_color="#B91C1C", height=40, width=100, font=ctk.CTkFont(weight="bold"),
command=self.stop_camera)
        self.stop_scan_btn.pack(side="right")

        # --- Right Card (Dashboard Analytics & Feed) ---
        self.right_card = ctk.CTkFrame(self.main_container, fg_color=self.colors["surface"], border_width=1,
```

```
border_color=self.colors["border"])
        self.right_card.pack(side="right", fill="both", expand=True, padx=(10, 0))

        ctk.CTkLabel(self.right_card, text="Daily Analytics", font=ctk.CTkFont(size=18, weight="bold")).pack(anchor="w", padx=20,
pady=(15, 5))

        self.kpi_grid = ctk.CTkFrame(self.right_card, fg_color="transparent")
        self.kpi_grid.pack(fill="x", padx=20, pady=10)
        self.kpi_grid.columnconfigure((0, 1), weight=1)

        self.kpi_total = self.create_kpi_card(self.kpi_grid, "Registered Total", "0", self.colors["blue"], 0, 0)
        self.kpi_present = self.create_kpi_card(self.kpi_grid, "Present Today", "0", self.colors["green"], 0, 1)
        self.kpi_absent = self.create_kpi_card(self.kpi_grid, "Absent Today", "0", self.colors["red"], 1, 0)
        self.kpi_rate = self.create_kpi_card(self.kpi_grid, "Attendance Rate %", "0%", self.colors["purple"], 1, 1)

        ctk.CTkLabel(self.right_card, text="Live Punch Stream", font=ctk.CTkFont(size=18, weight="bold")).pack(anchor="w", padx=20,
pady=(20, 5))

        self.stream_scroll = ctk.CTkScrollableFrame(self.right_card, fg_color="transparent")
        self.stream_scroll.pack(fill="both", expand=True, padx=20, pady=(0, 20))

        self.refresh_dashboard()

    def create_kpi_card(self, parent, title, value, color, row, col):
        card = ctk.CTkFrame(parent, fg_color=self.colors["bg_deep"], corner_radius=8, border_width=1, border_color=color)
        card.grid(row=row, column=col, sticky="nsew", padx=5, pady=5)
        ctk.CTkLabel(card, text=title, text_color=self.colors["text_muted"], font=ctk.CTkFont(size=12)).pack(anchor="w", padx=10,
pady=(10, 0))
        val_label = ctk.CTkLabel(card, text=value, text_color=color, font=ctk.CTkFont(size=24, weight="bold"))
        val_label.pack(anchor="w", padx=10, pady=(0, 10))
        return val_label

    def refresh_dashboard(self):
        stats = db.get_daily_attendance_stats()
        self.kpi_total.configure(text=str(stats['total']))
        self.kpi_present.configure(text=str(stats['present']))
        self.kpi_absent.configure(text=str(stats['absent']))
        self.kpi_rate.configure(text=f"{stats['percentage']}%")

        for widget in self.stream_scroll.winfo_children():
            widget.destroy()

        records = db.get_today_attendance()
        for rec in records:
            # rec = (user_code, name, date, time)
            card = ctk.CTkFrame(self.stream_scroll, fg_color=self.colors["bg_deep"], corner_radius=6, border_width=1,
border_color=self.colors["border"])
            card.pack(fill="x", pady=4)
            info_frame = ctk.CTkFrame(card, fg_color="transparent")
            info_frame.pack(side="left", padx=10, pady=10)
            ctk.CTkLabel(info_frame, text=rec[1], font=ctk.CTkFont(weight="bold", size=14)).pack(anchor="w")
            ctk.CTkLabel(info_frame, text=f"ID: {rec[0]}", font=ctk.CTkFont(size=12),
text_color=self.colors["text_muted"]).pack(anchor="w")

            badge = ctk.CTkFrame(card, fg_color=self.colors["green"], corner_radius=4)
            badge.pack(side="right", padx=10)
            ctk.CTkLabel(badge, text=rec[3], font=ctk.CTkFont(size=12, weight="bold"), text_color="#000000").pack(padx=8, pady=4)

    # --- KIOSK Camera Logic ---
    def start_kiosk_camera(self):
        if not self.camera_running:
            self.cap = cv2.VideoCapture(0)
            self.camera_running = True
            self.mode = "KIOSK"
            self.cam_badge.configure(text="● SCANNING ACTIVE", text_color=self.colors["green"])
            self.video_label.configure(text="")
            self.update_kiosk_camera()

    def update_kiosk_camera(self):
        if not self.camera_running or self.mode != "KIOSK":
            return

        ret, frame = self.cap.read()
        if ret:
            # Detect faces
            try:
                faces = self.face_engine.detect_faces(frame)
            except Exception:
                faces = None

            verified_info = None

            if faces is not None:
                for face in faces:
                    x, y, w, h = face[:4].astype(int)
                    # Expand bounding box slightly for the crop
```

```
                    pad_x = int(w * 0.1)
                    pad_y = int(h * 0.1)
                    x1 = max(0, x - pad_x)
                    y1 = max(0, y - pad_y)
                    x2 = min(frame.shape[1], x + w + pad_x)
                    y2 = min(frame.shape[0], y + h + pad_y)

                    face_crop = frame[y1:y2, x1:x2]

                    if face_crop.shape[0] > 0 and face_crop.shape[1] > 0:
                        try:
                            # Extract embedding
                            embedding = self.face_engine.extract_embedding(face_crop)

                            # Search in vector index (Updated threshold to 0.42)
                            uid, dist = self.vector_index.search(embedding, threshold=0.42)

                            if uid is not None:
                                user_info = db.get_user_by_id(int(uid))
                                if user_info:
                                    user_code, name = user_info
                                    verified_info = (user_code, name)

                                    # Draw HUD Box
                                    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
                                    cv2.putText(frame, f"{name} ({user_code})", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255,
0), 2)
                                else:
                                    # UID in index but not in DB?
                                    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
                                    cv2.putText(frame, "Unknown Face", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
                            else:
                                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
                                cv2.putText(frame, "Unknown Face", (x, y-10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
                        except Exception as e:
                            print(f"Embedding extraction error: {e}")

            # Convert to RGB and display
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            img = Image.fromarray(frame_rgb)
            ctk_img = ctk.CTkImage(light_image=img, dark_image=img, size=(frame.shape[1], frame.shape[0]))

            self.video_label.imgtk = ctk_img
            self.video_label.configure(image=ctk_img, text="")

            if verified_info:
                import sqlite3
                conn = sqlite3.connect(db.DB_PATH)
                cursor = conn.cursor()
                cursor.execute("SELECT * FROM attendance WHERE user_code = ? AND date = DATE('now', 'localtime')",
(verified_info[0],))
                already_marked = cursor.fetchone()
                conn.close()

                if already_marked:
                    self.cam_badge.configure(text=f"⚠ ALREADY MARKED: {verified_info[1]}", text_color="#F59E0B")
                    self.camera_running = False
                    self.after(2000, self.stop_camera)
                    return
                else:
                    # Mark attendance
                    success, timestamp = db.mark_attendance(verified_info[0], verified_info[1])
                    if success:
                        self.cam_badge.configure(text=f"✔ VERIFIED: {verified_info[1]}", text_color=self.colors["green"])
                        self.refresh_dashboard()

                        # Auto-Off Mechanism
                        self.camera_running = False
                        self.after(2000, self.stop_camera)
                        return
                    else:
                        self.cam_badge.configure(text=f"⚠ ALREADY MARKED: {verified_info[1]}", text_color="#F59E0B")
                        self.camera_running = False
                        self.after(2000, self.stop_camera)
                        return

        self.after(15, self.update_kiosk_camera)

    def stop_camera(self):
        self.camera_running = False
        if self.cap:
            self.cap.release()
            self.cap = None
        self.video_label.configure(image="", text="Camera is Idle.\nClick 'Start Attendance Scan' to begin")
        self.cam_badge.configure(text="● STANDBY", text_color=self.colors["text_muted"])
```

```
    def on_closing(self):
        self.stop_camera()
        self.destroy()

    # --- Admin Authentication ---
    def open_admin_auth(self):
        if hasattr(self, 'admin_window') and self.admin_window is not None and self.admin_window.winfo_exists():
            self.admin_window.focus()
            return

        dialog = ctk.CTkToplevel(self)
        dialog.title("Admin Authentication")
        dialog.geometry("320x350")
        dialog.transient(self)
        dialog.grab_set()
        dialog.configure(fg_color=self.colors["bg_deep"])

        ctk.CTkLabel(dialog, text="🔓 Admin Portal", font=ctk.CTkFont(size=20, weight="bold")).pack(pady=(30, 20))

        user_entry = ctk.CTkEntry(dialog, placeholder_text="Username", width=200)
        user_entry.pack(pady=10)

        pass_entry = ctk.CTkEntry(dialog, placeholder_text="Password", show="*", width=200)
        pass_entry.pack(pady=10)

        def attempt_login():
            if db.verify_admin(user_entry.get(), pass_entry.get()):
                dialog.destroy()
                self.stop_camera()
                self.open_admin_dashboard()
            else:
                messagebox.showerror("Auth Error", "Invalid Credentials", parent=dialog)

        ctk.CTkButton(dialog, text="Login", fg_color=self.colors["blue"], hover_color="#1E3A8A", command=attempt_login,
width=200).pack(pady=(20, 10))

    # --- Admin Dashboard ---
    def open_admin_dashboard(self):
        self.admin_window = ctk.CTkToplevel(self)
        self.admin_window.title("Admin Dashboard - AURA-FACE")
        self.admin_window.geometry("1000x700")
        self.admin_window.minsize(1000, 700)
        self.admin_window.transient(self)
        self.admin_window.grab_set()
        self.admin_window.configure(fg_color=self.colors["bg_deep"])

        # Header
        header = ctk.CTkFrame(self.admin_window, fg_color=self.colors["surface"], corner_radius=0, height=60)
        header.pack(fill="x", side="top")
        ctk.CTkLabel(header, text="Admin Control Panel", font=ctk.CTkFont(size=20, weight="bold")).pack(side="left", padx=20,
pady=15)

        ctk.CTkButton(header, text="Return to Kiosk", fg_color="transparent", border_width=1, border_color=self.colors["text_muted"],
                      command=self.admin_window.destroy).pack(side="right", padx=20, pady=15)

        # Tabview
        self.admin_tabs = ctk.CTkTabview(self.admin_window, fg_color=self.colors["bg_deep"])
        self.admin_tabs.pack(fill="both", expand=True, padx=20, pady=20)

        self.admin_tabs.add("User Management & Enrollment")
        self.admin_tabs.add("Attendance Analytics & Export")
        self.admin_tabs.add("Daily & Monthly Reports")

        self.setup_enrollment_tab()
        self.setup_analytics_tab()
        self.setup_reports_tab()

    def setup_enrollment_tab(self):
        tab = self.admin_tabs.tab("User Management & Enrollment")

        # Enrollment Form
        form_frame = ctk.CTkFrame(tab, fg_color=self.colors["surface"], corner_radius=8, border_width=1,
border_color=self.colors["border"])
        form_frame.pack(fill="x", pady=(0, 20))

        ctk.CTkLabel(form_frame, text="User ID / Roll No:").grid(row=0, column=0, padx=10, pady=(15, 5), sticky="w")
        self.reg_id_entry = ctk.CTkEntry(form_frame, width=150)
        self.reg_id_entry.grid(row=1, column=0, padx=10, pady=(0, 15))

        ctk.CTkLabel(form_frame, text="Full Name:").grid(row=0, column=1, padx=10, pady=(15, 5), sticky="w")
        self.reg_name_entry = ctk.CTkEntry(form_frame, width=200)
        self.reg_name_entry.grid(row=1, column=1, padx=10, pady=(0, 15))

        ctk.CTkLabel(form_frame, text="Registration Date:").grid(row=0, column=2, padx=10, pady=(15, 5), sticky="w")
        self.reg_date_entry = ctk.CTkEntry(form_frame, width=150)
        self.reg_date_entry.insert(0, time.strftime("%Y-%m-%d"))
```

```
        self.reg_date_entry.grid(row=1, column=2, padx=10, pady=(0, 15))

        self.enroll_btn = ctk.CTkButton(form_frame, text="Enroll Face Biometrics", fg_color=self.colors["purple"],
hover_color="#7C3AED", command=self.start_enrollment_process)
        self.enroll_btn.grid(row=1, column=3, padx=20, pady=(0, 15))

        self.enroll_status = ctk.CTkLabel(form_frame, text="", text_color=self.colors["green"])
        self.enroll_status.grid(row=2, column=0, columnspan=4, pady=(0, 10))

        # Enrollment Camera View (Hidden by default)
        self.enroll_cam_frame = ctk.CTkFrame(tab, fg_color="transparent")
        self.enroll_cam_label = ctk.CTkLabel(self.enroll_cam_frame, text="")
        self.enroll_cam_label.pack()

        # Table View
        self.users_textbox = ctk.CTkTextbox(tab, fg_color=self.colors["surface"], border_width=1, border_color=self.colors["border"],
font=ctk.CTkFont(family="Consolas", size=13))
        self.users_textbox.pack(fill="both", expand=True, pady=(0, 10))
        self.refresh_users_table()

        # User Management View
        manage_frame = ctk.CTkFrame(tab, fg_color=self.colors["surface"], corner_radius=8, border_width=1,
border_color=self.colors["border"])
        manage_frame.pack(fill="x", pady=(0, 10))

        ctk.CTkLabel(manage_frame, text="Manage User (Enter UID):").grid(row=0, column=0, padx=10, pady=10, sticky="w")
        self.manage_uid_entry = ctk.CTkEntry(manage_frame, width=80)
        self.manage_uid_entry.grid(row=0, column=1, padx=10, pady=10)

        ctk.CTkButton(manage_frame, text="Edit Details", fg_color=self.colors["blue"],
command=self.open_edit_user_dialog).grid(row=0, column=2, padx=10, pady=10)
        ctk.CTkButton(manage_frame, text="Delete User", fg_color=self.colors["red"], command=self.delete_user_action).grid(row=0,
column=3, padx=10, pady=10)

    def refresh_users_table(self):
        if not hasattr(self, 'users_textbox'): return
        self.users_textbox.configure(state="normal")
        self.users_textbox.delete("1.0", "end")
        self.users_textbox.insert("end", f"{'S.No':<5} | {'UID':<6} | {'User Code':<15} | {'Full Name':<25} | {'Reg Date'}\n")
        self.users_textbox.insert("end", "-"*80 + "\n")
        users = db.get_all_users()
        for i, u in enumerate(users, start=1):
            self.users_textbox.insert("end", f"{i:<5} | {u[0]:<6} | {u[1]:<15} | {u[2]:<25} | {u[3]}\n")
        self.users_textbox.configure(state="disabled")

    def open_edit_user_dialog(self):
        uid_str = self.manage_uid_entry.get().strip()
        if not uid_str.isdigit():
            messagebox.showerror("Error", "Please enter a valid numeric UID")
            return

        uid = int(uid_str)
        user = db.get_user_by_id(uid)
        if not user:
            messagebox.showerror("Error", "User not found")
            return

        dialog = ctk.CTkToplevel(self.admin_window)
        dialog.title("Edit User")
        dialog.geometry("350x300")
        dialog.transient(self.admin_window)
        dialog.grab_set()

        ctk.CTkLabel(dialog, text=f"Editing User UID: {uid}", font=ctk.CTkFont(size=16, weight="bold")).pack(pady=20)

        code_entry = ctk.CTkEntry(dialog, placeholder_text="User Code", width=200)
        code_entry.insert(0, user[0])
        code_entry.pack(pady=10)

        name_entry = ctk.CTkEntry(dialog, placeholder_text="Full Name", width=200)
        name_entry.insert(0, user[1])
        name_entry.pack(pady=10)

        # Fetch reg_date from DB
        user_full = next((u for u in db.get_all_users() if u[0] == uid), None)
        reg_date = user_full[3] if user_full else ""

        date_entry = ctk.CTkEntry(dialog, placeholder_text="Registration Date", width=200)
        date_entry.insert(0, reg_date)
        date_entry.pack(pady=10)

        def save_changes():
            success, msg = db.update_user(uid, code_entry.get().strip(), name_entry.get().strip(), date_entry.get().strip())
            if success:
                messagebox.showinfo("Success", "User updated successfully", parent=dialog)
                dialog.destroy()
```

```
                self.refresh_users_table()
                self.manage_uid_entry.delete(0, 'end')
            else:
                messagebox.showerror("Error", msg, parent=dialog)

        ctk.CTkButton(dialog, text="Save Changes", command=save_changes, fg_color=self.colors["green"]).pack(pady=20)

    def delete_user_action(self):
        uid_str = self.manage_uid_entry.get().strip()
        if not uid_str.isdigit():
            messagebox.showerror("Error", "Please enter a valid numeric UID")
            return

        uid = int(uid_str)
        user = db.get_user_by_id(uid)
        if not user:
            messagebox.showerror("Error", "User not found")
            return

        if messagebox.askyesno("Confirm Delete", f"Are you sure you want to delete user {user[1]} ({user[0]})?\nThis will also remove
their attendance and biometric data."):
            success, msg = db.delete_user(uid)
            if success:
                self.vector_index.remove_vector(uid)
                messagebox.showinfo("Success", "User deleted successfully")
                self.refresh_users_table()
                self.manage_uid_entry.delete(0, 'end')
            else:
                messagebox.showerror("Error", msg)


    def start_enrollment_process(self):
        code = self.reg_id_entry.get().strip()
        name = self.reg_name_entry.get().strip()
        reg_date = self.reg_date_entry.get().strip()

        if not code or not name:
            messagebox.showerror("Error", "Please fill in User ID and Name")
            return

        # Register in DB
        numeric_id = db.add_user(code, name, reg_date)
        if numeric_id == -1:
            messagebox.showerror("Error", "User Code already exists!")
            return

        self.enrollment_user_id = numeric_id
        self.enrollment_crops = []

        self.enroll_status.configure(text="Please look at the camera. Capturing 5 face samples...", text_color=self.colors["blue"])
        self.enroll_btn.configure(state="disabled")

        self.enroll_cam_frame.pack(fill="x", pady=10)
        self.users_textbox.pack_forget()

        self.mode = "ENROLL"
        self.cap = cv2.VideoCapture(0)
        self.camera_running = True
        self.update_enrollment_camera()

    def update_enrollment_camera(self):
        if not self.camera_running or self.mode != "ENROLL":
            return

        ret, frame = self.cap.read()
        if ret:
            try:
                faces = self.face_engine.detect_faces(frame)
            except Exception:
                faces = None

            if faces is not None and len(faces) > 0:
                # Use the largest face
                face = max(faces, key=lambda f: f[2]*f[3])
                x, y, w, h = face[:4].astype(int)

                pad_x = int(w * 0.1)
                pad_y = int(h * 0.1)
                x1 = max(0, x - pad_x)
                y1 = max(0, y - pad_y)
                x2 = min(frame.shape[1], x + w + pad_x)
                y2 = min(frame.shape[0], y + h + pad_y)

                face_crop = frame[y1:y2, x1:x2]
                if face_crop.shape[0] > 0 and face_crop.shape[1] > 0:
                    embedding = self.face_engine.extract_embedding(face_crop)
```

```
                    self.enrollment_crops.append(embedding)

                    cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
                    cv2.putText(frame, f"Sample {len(self.enrollment_crops)}/{self.enrollment_target}", (x, y-10),
cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

            # Show preview
            frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            img = Image.fromarray(frame_rgb)
            ctk_img = ctk.CTkImage(light_image=img, dark_image=img, size=(640, 480))
            self.enroll_cam_label.configure(image=ctk_img)

            if len(self.enrollment_crops) >= self.enrollment_target:
                self.finish_enrollment()
                return

        # 300ms delay to capture different expressions
        self.after(300, self.update_enrollment_camera)

    def finish_enrollment(self):
        self.camera_running = False
        if self.cap:
            self.cap.release()
            self.cap = None

        self.enroll_cam_frame.pack_forget()
        self.users_textbox.pack(fill="both", expand=True)
        self.enroll_btn.configure(state="normal")

        # Calculate mean embedding
        mean_emb = np.mean(self.enrollment_crops, axis=0).astype(np.float32)
        # Re-normalize
        norm = np.linalg.norm(mean_emb)
        if norm > 0:
            mean_emb = mean_emb / norm

        # Check for duplicate face before adding
        try:
            uid, dist = self.vector_index.search(mean_emb, threshold=0.15)
            if dist is not None:
                print(f"[Enrollment Check] Closest existing vector distance: {dist:.4f} (Threshold: 0.15)")

            if uid is not None:
                # Duplicate found! Rollback the DB entry
                db.delete_user(self.enrollment_user_id)
                self.enroll_status.configure(text=f"Error: Face already registered to ID {uid}", text_color=self.colors["red"])
                self.refresh_users_table()
                return
        except Exception as e:
            print(f"Error checking for duplicates: {e}")

        # Add to FAISS Index
        try:
            self.vector_index.add_vector(self.enrollment_user_id, mean_emb)
            self.enroll_status.configure(text="✔ User Biometrics Enrolled Successfully!", text_color=self.colors["green"])
            self.reg_id_entry.delete(0, 'end')
            self.reg_name_entry.delete(0, 'end')
        except Exception as e:
            self.enroll_status.configure(text=f"Error saving to index: {e}", text_color=self.colors["red"])

        self.refresh_users_table()

    def setup_analytics_tab(self):
        tab = self.admin_tabs.tab("Attendance Analytics & Export")

        ctrl_frame = ctk.CTkFrame(tab, fg_color="transparent")
        ctrl_frame.pack(fill="x", pady=(0, 10))

        ctk.CTkButton(ctrl_frame, text="Export Today's Sheet (CSV)", command=self.export_today_csv,
fg_color=self.colors["blue"]).pack(side="left", padx=5)
        ctk.CTkButton(ctrl_frame, text="Export Summary (CSV)", command=self.export_summary_csv,
fg_color=self.colors["purple"]).pack(side="left", padx=5)

        self.analytics_textbox = ctk.CTkTextbox(tab, fg_color=self.colors["surface"], border_width=1,
border_color=self.colors["border"], font=ctk.CTkFont(family="Consolas", size=13))
        self.analytics_textbox.pack(fill="both", expand=True)

        # Load Analytics Data
        self.refresh_analytics_table()

    def refresh_analytics_table(self):
        if not hasattr(self, 'analytics_textbox'): return
        summary = db.get_user_attendance_summary()
        self.analytics_textbox.configure(state="normal")
        self.analytics_textbox.delete("1.0", "end")
        self.analytics_textbox.insert("end", f"{'User Code':<15} | {'Name':<25} | {'Total Days':<12} | {'Present':<10} | {'Absent':
```

```
<10} | {'Attendance %'}\n")
        self.analytics_textbox.insert("end", "-"*95 + "\n")
        for s in summary:
            self.analytics_textbox.insert("end", f"{s['user_code']:<15} | {s['name']:<25} | {s['total_days']:<12} | {s['present']:
<10} | {s['absent']:<10} | {s['percentage']}%\n")
        self.analytics_textbox.configure(state="disabled")

    def export_today_csv(self):
        records = db.get_today_attendance()
        if not records:
            messagebox.showinfo("Info", "No attendance records today.")
            return

        today = time.strftime("%Y-%m-%d")
        path = filedialog.asksaveasfilename(defaultextension=".csv", initialfile=f"Attendance_{today}.csv")
        if path:
            with open(path, 'w', newline='') as f:
                w = csv.writer(f)
                w.writerow(["User Code", "Name", "Date", "Time"])
                w.writerows(records)
            messagebox.showinfo("Success", f"Exported to {path}")

    def export_summary_csv(self):
        summary = db.get_user_attendance_summary()
        if not summary:
            messagebox.showinfo("Info", "No data to export.")
            return

        today = time.strftime("%Y-%m-%d")
        path = filedialog.asksaveasfilename(defaultextension=".csv", initialfile=f"Attendance_Summary_{today}.csv")
        if path:
            with open(path, 'w', newline='') as f:
                w = csv.writer(f)
                w.writerow(["User Code", "Name", "Total Days", "Present", "Absent", "Attendance %"])
                for s in summary:
                    w.writerow([s['user_code'], s['name'], s['total_days'], s['present'], s['absent'], s['percentage']])
            messagebox.showinfo("Success", f"Exported to {path}")

    def setup_reports_tab(self):
        tab = self.admin_tabs.tab("Daily & Monthly Reports")

        # Controls Frame
        ctrl_frame = ctk.CTkFrame(tab, fg_color="transparent")
        ctrl_frame.pack(fill="x", pady=(0, 10))

        # View Mode Switch
        self.report_mode = ctk.StringVar(value="Daily")

        ctk.CTkRadioButton(ctrl_frame, text="Daily Report", variable=self.report_mode, value="Daily",
                           command=self.refresh_reports_table).pack(side="left", padx=10)
        ctk.CTkRadioButton(ctrl_frame, text="Monthly Report", variable=self.report_mode, value="Monthly",
                           command=self.refresh_reports_table).pack(side="left", padx=10)

        # Date Input
        ctk.CTkLabel(ctrl_frame, text="Date (YYYY-MM-DD):").pack(side="left", padx=(20, 5))
        self.report_date_entry = ctk.CTkEntry(ctrl_frame, width=120)
        self.report_date_entry.insert(0, time.strftime("%Y-%m-%d"))
        self.report_date_entry.pack(side="left", padx=5)

        # Month Input
        ctk.CTkLabel(ctrl_frame, text="Month (YYYY-MM):").pack(side="left", padx=(20, 5))
        self.report_month_entry = ctk.CTkEntry(ctrl_frame, width=100)
        self.report_month_entry.insert(0, time.strftime("%Y-%m"))
        self.report_month_entry.pack(side="left", padx=5)

        ctk.CTkButton(ctrl_frame, text="Apply Filter", command=self.refresh_reports_table,
fg_color=self.colors["blue"]).pack(side="left", padx=10)

        # Export Button
        ctk.CTkButton(ctrl_frame, text="Export CSV", command=self.export_reports_csv,
fg_color=self.colors["green"]).pack(side="right", padx=10)

        # Treeview for Table
        table_frame = ctk.CTkFrame(tab)
        table_frame.pack(fill="both", expand=True, pady=10)

        # Treeview style
        style = ttk.Style()
        style.theme_use("default")
        style.configure("Treeview",
                        background=self.colors["surface"],
                        foreground="white",
                        fieldbackground=self.colors["surface"],
                        borderwidth=0,
                        rowheight=25)
        style.map('Treeview', background=[('selected', self.colors["blue"])])
```

```
        style.configure("Treeview.Heading", background=self.colors["bg_deep"], foreground="white", borderwidth=1)

        self.report_tree = ttk.Treeview(table_frame, show="headings")
        self.report_tree.pack(side="left", fill="both", expand=True)

        scrollbar = ttk.Scrollbar(table_frame, orient="vertical", command=self.report_tree.yview)
        scrollbar.pack(side="right", fill="y")
        self.report_tree.configure(yscrollcommand=scrollbar.set)

        self.report_data = [] # Store data for export
        self.refresh_reports_table()

    def refresh_reports_table(self):
        if not hasattr(self, 'report_tree'): return

        # Clear existing
        for item in self.report_tree.get_children():
            self.report_tree.delete(item)

        mode = self.report_mode.get()
        if mode == "Daily":
            self.report_tree["columns"] = ("code", "name", "date", "time", "status")
            self.report_tree.heading("code", text="User Code")
            self.report_tree.heading("name", text="Name")
            self.report_tree.heading("date", text="Date")
            self.report_tree.heading("time", text="Punch Time")
            self.report_tree.heading("status", text="Status")

            self.report_tree.column("code", width=100)
            self.report_tree.column("name", width=200)
            self.report_tree.column("date", width=100)
            self.report_tree.column("time", width=100)
            self.report_tree.column("status", width=100)

            date_str = self.report_date_entry.get().strip()
            if not date_str: date_str = time.strftime("%Y-%m-%d")

            report = db.get_daily_report(date_str)
            self.report_data = report.get("records", [])

            for r in self.report_data:
                self.report_tree.insert("", "end", values=(r["user_code"], r["name"], r["date"], r["punch_time"], r["status"]))

        else: # Monthly
            self.report_tree["columns"] = ("code", "name", "total", "present", "absent", "rate")
            self.report_tree.heading("code", text="User Code")
            self.report_tree.heading("name", text="Name")
            self.report_tree.heading("total", text="Total Days")
            self.report_tree.heading("present", text="Present")
            self.report_tree.heading("absent", text="Absent")
            self.report_tree.heading("rate", text="Attendance %")

            self.report_tree.column("code", width=100)
            self.report_tree.column("name", width=200)
            self.report_tree.column("total", width=80)
            self.report_tree.column("present", width=80)
            self.report_tree.column("absent", width=80)
            self.report_tree.column("rate", width=100)

            month_str = self.report_month_entry.get().strip()
            if not month_str: month_str = time.strftime("%Y-%m")

            report = db.get_monthly_report(month_str)
            self.report_data = report.get("records", [])

            for r in self.report_data:
                self.report_tree.insert("", "end", values=(r["user_code"], r["name"], r["total_days"], r["days_present"],
r["days_absent"], f"{r['attendance_percentage']}%"))

    def export_reports_csv(self):
        if not hasattr(self, 'report_data') or not self.report_data:
            messagebox.showinfo("Info", "No data to export.")
            return

        mode = self.report_mode.get()
        if mode == "Daily":
            date_str = self.report_date_entry.get().strip() or time.strftime("%Y-%m-%d")
            path = filedialog.asksaveasfilename(defaultextension=".csv", initialfile=f"Daily_Report_{date_str}.csv")
            if path:
                with open(path, 'w', newline='') as f:
                    w = csv.writer(f)
                    w.writerow(["User Code", "Name", "Date", "Punch Time", "Status"])
                    for r in self.report_data:
                        w.writerow([r["user_code"], r["name"], r["date"], r["punch_time"], r["status"]])
                messagebox.showinfo("Success", f"Exported to {path}")
        else:
```

```
            month_str = self.report_month_entry.get().strip() or time.strftime("%Y-%m")
            path = filedialog.asksaveasfilename(defaultextension=".csv", initialfile=f"Monthly_Report_{month_str}.csv")
            if path:
                with open(path, 'w', newline='') as f:
                    w = csv.writer(f)
                    w.writerow(["User Code", "Name", "Total Days", "Present", "Absent", "Attendance %"])
                    for r in self.report_data:
                        w.writerow([r["user_code"], r["name"], r["total_days"], r["days_present"], r["days_absent"], f"
{r['attendance_percentage']}%"])
                messagebox.showinfo("Success", f"Exported to {path}")

if __name__ == "__main__":
    app = BiometricDashboard()
    app.mainloop()
```

Explanation:

- **Local Integration**: Directly imports `db_manager`, `VectorIndex`, and `FaceEngine`, bypassing the FastAPI layer entirely.
- **Threading/Loops**: Utilizes Tkinter's `.after()` loop to continuously poll the local webcam via OpenCV `VideoCapture(0)` at ~15ms intervals, preventing UI blocking.
- **Enrollment Flow**: Collects 5 frames using an internal state counter, calculates the mean normalized embedding, verifies the 0.15 collision threshold, and commits directly.
- **Treeview Analytics**: Utilizes `ttk.Treeview` alongside `csv.writer` to populate tabular data and export local files via `filedialog`.

---

3. END-TO-END SCAN & DATALIFECYCLES

Live Scan Pipeline

1. **Capture**: React `Webcam` or OpenCV grabs a standard frame.
2. **Transmission**: Frame is converted to Base64 (web) or kept in memory (desktop).
3. **Detection**: OpenCV YuNet ONNX model localizes the face bounding box.
4. **Extraction**: MobileNetV2 crops the face, runs a forward pass, and outputs a 128-d vector.
5. **Normalization**: L2 normalization transforms the vector onto a hypersphere surface.
6. **Similarity Search**: FAISS calculates distance against the index. If top-1 distance <= 0.55 (FastAPI) or 0.42 (Desktop), a UID is yielded.
7. **Verification**: Backend queries SQLite to ensure `UNIQUE(user_code, date)` hasn't been violated today.
8. **Feedback**: HUD updates reticle color (Green=Success, Amber=Duplicate, Red=Unknown) and pushes log payload to the live feed array.

Edit / Delete Lifecycles

- **Edit**: User edits name/code -> PUT payload -> SQLite `users` table updated -> Cascading update targets `attendance` table, aligning all historical logs to the new `user_code` -> Return success.
- **Delete**: User confirms delete -> DELETE payload -> FAISS `remove_vector(UID)` purges embedding -> SQLite purges `users` record -> SQLite purges associated `attendance` records -> Vector index saved to disk -> Return success -> Frontend invalidates local arrays and refetches.

CSV Export Pipeline

- **Web**: Endpoints like `/api/export/today` generate an `io.StringIO` CSV buffer and return it as a `text/csv` stream, prompting an immediate download. Alternatively, client-side blob generation parses React state arrays directly into downloadable CSVs.
- **Desktop**: `csv.writer` targets absolute local file paths selected via `filedialog.asksaveasfilename()`.

---

4. STEP-BY-STEP PDF EXPORT INSTRUCTIONS

To export this documentation into a formatted PDF:

Method A: Visual Studio Code

1. Install the extension: **Markdown PDF** by yzane.
2. Open this `SYSTEM_DOCUMENTATION.md` file in VS Code.
3. Right-click anywhere in the editor.
4. Select **Markdown PDF: Export (pdf)**.
5. The PDF will be generated instantly in the same directory.

Method B: Chrome / Edge Browser

1. Open this Markdown file using a Markdown Viewer extension in Chrome/Edge, or render it on GitHub/GitLab.
2. Press `CTRL + P` (or `CMD + P` on Mac).
3. Under "Destination", select **Save as PDF**.
4. Disable "Headers and footers" and enable "Background graphics".
5. Click **Save**.